



CSE 6234 Software Design Term 2610
Group 1 TT3L

Ecommerce System (MyGrocer)

Lecturer: Chang Ming Man

STUDENT NAME	STUDENT ID
QAISARA HANNAH BINTI KAMARUL ARIFFIN	243UC245DK
NURALIAH ATHIRAH BINTI USMAN	243UC247LW
MUHAMMAD NAUFAL BIN FAUZI	243UC245JM

Acknowledgement

We, the members of Group 1 from TT3L, would like to express our deepest gratitude to everyone who contributed to the successful completion of our Assignment for CSE 6234 Software Design. ***MYGROCER: An Online Grocery E-Commerce System.***

Our sincere thanks go to our lecturer, ***Mr. Chan Ming Man***, for the invaluable guidance, technical insights, and continuous support provided throughout the development of this documentation. We also appreciate the collaborative efforts of all team members, whose dedication to analysing system requirements and designing the ***MYGROCER*** framework was essential to this project's success. This collaboration not only facilitated the completion of this documentation but also enhanced our mutual understanding of software design concepts, design principles, design patterns, and system evaluation.

In conclusion, we are deeply thankful to everyone who played a role in bringing this project to fruition as your contributions have been invaluable and we are sincerely appreciative of the support received throughout this journey.

Table of Contents

Acknowledgement	2
Abstract (Executive summary).....	1
1.0 Introduction	1
1.1. Abstract.....	1
1.2. Problem Statement.....	2
1.3. Project Objectives	2
1.4 Literature Review	3
1.5 Project and System Scope	4
1.5.1 Project Scope	4
1.5.2 System Scope	7
2.0 System Overview	23
2.1 Generic Use Case.....	23
2.2 Class Diagrams.....	24
2.3 ERD	25
2.4 Object Diagrams	26
3.0 Requirements	27
3.1 Software Design Concepts.....	27
3.1.1 Abstraction	27
3.1.3 Modularity	34
3.1.4 Functional Independence	35
3.1.5 Refinement.....	43
3.1.6 Refactoring	44
3.2.7 Architecture.....	48
3.1.8 Pattern.....	50
3.2.1 Single Responsibility Principle (SRP)	51
3.2.2 Open / Closed Principle (OCP).....	55
3.2.3 Liskov Substitution Principle (LSP).....	57
3.2.4 Interface Segregation Principle	62
3.2.5 Dependency Inversion Principle (DIP).....	64
4.0 Proposed Design Patterns	68
4.1 Singleton Pattern.....	68
4.2 Factory Pattern	75
4.3 Observer Pattern.....	80
5. Conclusion and suggestion	84
5.1 Conclusion	84
5.2 Suggestions for Future Enhancement	84

6. Bibliography/Reference	86
--	-----------

Abstract (Executive summary)

1.0 Introduction

1.1. Abstract

MYGROCER is a scalable web-based platform designed to digitize the affordable Malaysian hypermarket experience, drawing inspiration from **MYDIN's** competitive pricing and diverse product range. The system provides a seamless shopping journey where users can browse categorized essentials from fresh produce to household goods and manage a virtual cart. It supports various fulfilment methods, such as delivery and self-collection, alongside secure payment options like FPX, cards, and COD, while providing real-time order tracking from confirmation to fulfilment.

From a software design perspective, **MYGROCER** is architected using three key design patterns:

1. **Singleton Pattern** manages the database connection, ensuring a single shared instance is supported throughout the application lifecycle, preventing resource conflicts.
2. **Factory Pattern** handles the dynamic creation of different product category objects (e.g., Fresh Produce, Packaged Goods, Beverages, Household) without exposing instantiation logic to client code.
3. **Observer Pattern** powers real-time order status notifications, automatically informing customers and fulfilment staff when an order's status changes.

The system is built on well-established Software Design Concepts such as abstraction, encapsulation, modularity, and functional independence, and adheres to SOLID principles to ensure maintainability, scalability, and testability.

The system architecture further adheres to core software design principles including the Single Responsibility Principle (SRP), Open/Closed Principle (OCP), and Dependency Inversion Principle (DIP), ensuring the codebase remains maintainable, extensible, and testable as the system scales

The proposed system covers user registration and login, product browsing and search, shopping cart management, multiple payment method support, order tracking, and an admin back-office for inventory and order management.

MYGROCER represents a practical and technically sound solution for the growing demand for online grocery shopping in Malaysia, bridging the gap between physical retail affordability and digital convenience.

1.2. Problem Statement

1. **Complexity of Pricing Logic:** Managing different price tiers (Wholesale, Member, and Retail) manually creates high maintenance overhead and risks of calculation errors during checkout.
2. **Inventory Synchronization Latency:** The inability to sync real-time stock levels between the physical hypermarket shelves and the online store leads to "ghost orders" (customers buying items that are actually out of stock).
3. **Inaccessibility for Time-Constrained Consumers:** Working adults and families in urban areas often face difficulty visiting physical grocery stores during operational hours, particularly during peak traffic periods in cities such as Kuala Lumpur, Penang, and Johor Bahru.
4. **Notification Delays:** Manual systems for notifying procurement managers about low stock or customers about back-ordered items lead to supply chain bottlenecks and poor user experience.
5. **High Coupling in Payment Systems:** A rigid software architecture makes it difficult to integrate new payment gateways (like e-wallets or BNPL services) without breaking existing checkout code.

1.3. Project Objectives

- **To design a scalable e-commerce platform** that replicates the product diversity and affordability of a Malaysian hypermarket in a digital environment for urban consumers.
- **To implement a dynamic pricing and inventory module** that automates price tier calculations and ensures real-time stock synchronization to eliminate manual errors.
- **To apply the Singleton Design Pattern** to the database connection layer to ensure efficient, conflict-free resource management across all system modules.
- **To apply the Factory Design Pattern** for product instantiation, allowing the system to handle diverse categories (Fresh Produce, Household, etc.) without modifying core code.
- **To apply the Strategy Design Pattern** to manage complex pricing tiers (Retail, Wholesale, Member), ensuring the checkout logic remains maintainable and extensible.
- **To apply the Observer Design Pattern** to the order management module to enable automated, real-time status notifications for both customers and fulfillment staff.
- **To integrate a decoupled payment gateway system** supporting FPX, cards, and COD, ensuring the design adheres to **SOLID principles** for easy future expansion (e.g., e-wallets).
- **To develop a multi-role interface** for Customers and Administrators with clearly defined permissions to streamline store management and the user shopping journey.

1.4 Literature Review

1. Adoption of online grocery shopping is heavily influenced by factors like convenience, usability, and technology acceptance. (Prabowo et al, 2020) highlighted that online grocery platforms must provide efficient navigation, secure transaction systems, and reliable service quality to encourage continuous user adoption. The study also emphasized the importance of scalable and maintainable software systems to support future enhancements and integrations within e-commerce platforms. These findings support the development of MYGROCER as a user-friendly and scalable grocery e-commerce platform.
2. (Moorthy et al, 2025) studied online grocery shopping behaviour in Malaysia by integrating the Technology Acceptance Model (TAM) and Theory of Planned Behaviour (TPB). The researchers found that consumer attitudes, perceived behavioural control, and social influence significantly affect the intention to use online grocery platforms. The study concluded that Malaysian consumers are increasingly accepting online grocery shopping due to convenience and time-saving benefits. This research demonstrates the growing demand for online grocery services in Malaysia and supports the relevance of developing MYGROCER to address modern consumer needs.
3. Eriksson and Stenius (2023) discussed customer expectations and service quality improvements for online grocery systems. Their study revealed that customers value accurate order fulfilment, real-time product availability, delivery reliability, and efficient customer service. The researchers also emphasized the importance of flexible and efficient system design in maintaining customer satisfaction and improving overall service quality. These findings are relevant to MYGROCER, particularly in implementing real-time inventory synchronization, order tracking, and maintainable software architecture through the application of software design principles and design patterns.

1.5 Project and System Scope

1.5.1 Project Scope

Aspect	Details
Project Name	MYGROCER
Objective	To develop a scalable web-based online grocery e-commerce platform that replicates the affordability and product diversity of a Malaysian hypermarket, while enabling customers to conveniently shop, pay, and track orders digitally.
Primary Users	Customers and Admin
Key Features	Customers: 1. Registration account: -Customers can create a personal account by submitting their full name, email, phone number, and password. The system validates inputs and stores credentials securely in the database. 2. Login and Logout account: -Registered customers can log in using their email and password to access restricted features such as cart management and order placement, and log out to terminate their session. 3. Browse the item: -Customers can browse products organized by categories (e.g., Fresh Produce, Packaged Goods, Beverages, Household). Each item displays its name, price, stock status, and product image. 4. Search for item: -Customers can use the search bar to find specific products by name or keyword. The system returns matching results from the product catalogue. 5. Manage carts: -Customers can add items to a virtual shopping cart, update quantities, remove items, and view a running total before proceeding to checkout. 6. Place order: -Customers select their preferred fulfilment method (delivery or self-collection), confirm delivery address or collection point, and submit their order. 7. Make payment: -Customers can complete purchases using supported payment methods: FPX, Debit/Credit Card, or Cash on Delivery (COD). The system processes payment and confirms the transaction. 8. Track order: -After placing an order, customers can monitor their order status in real time (e.g., Order Confirmed → Processing → Out for Delivery → Delivered).

	Admin: 1. Track system Functionality: -Admin can monitor overall system activity including login/logout events, sales data, user feedback, inventory levels, and generated reports to ensure operational health. 2. View Item: -Admin can view the full product catalogue including item details such as name, category, price, stock quantity, and product image. 3. Manage Items: -ADD: Admin can add new products to the catalogue by entering item name, category, price, description, stock quantity, and uploading a product image. -DELETE: Admin can permanently remove a product from the catalogue. The system prompts for confirmation before deletion to prevent accidental removal. -UPDATE: Admin can edit existing product details such as name, price, stock quantity, description, or product image to keep the catalogue accurate and up to date. 4. View Order: -Admin can view all customer orders, including order details, customer information, items ordered, total payment, fulfilment method, and current order status. Admin can update order status to notify customers in real time.
System Boundary	The system includes the following: 1. Customer-facing web interface for browsing, cart management, checkout, and order tracking. 2. Admin back-office panel for inventory and order management. 3. Database layer managing user accounts, product catalogue, cart data, and orders. 4. Payment processing module supporting FPX, Card and COD 5. Notification module for real-time order status updates.
Out of scope	Future Implementation : As suggestion -Third-party e-wallet integration (e.g., Touch'n Go, GrabPay) -Loyalty/rewards points system -Mobile application (Android/iOS) -Multi-vendor/marketplace support -AI- driven product recommendations

Project Specification	Hardware Requirements for Owner/Customer: Web server with minimum 8GB RAM, multi-core processor, SSD storage. Hardware Requirements for Customer: Any device with a modern web browser and internet connection. <u>Software</u> -Front End: React/Vue -Back End: asp.net -Database: SQLite -Operating System: Windows Server/Linux -Programming Language: JavaScript, HTML, CSS,C#.
------------------------------	---

1.5.2 System Scope

1.5.2.1 Customer/User

1.5.2.1.1 Register Account

Use Case	01
Use Case Name	Register account
Actor	Customer/User
Description	This use case enables users to create an account in the system to have access to features that are restricted to registered users which are adding product to cart, making purchases and tracking orders.
Pre-condition	1. The user must be able to load the system and have access to the system on their device. 2. The registration page can be found and is functional 3. The system must be connected to the database to be able to store the user's credentials securely.
Basic Course Flow	1. The user navigates through the system to find and click the "Register Account" button which will lead to the registration page. 2. The user enters and submits the following information: I. Full name II. Email address III. Phone number: IV. Password 3. The system validates the input submitted. 4. If the validations are successful, the system securely stores the information in the database. 5. The system confirms successful registration and redirects users to the login page.
Post-Condition	New account is successfully made and stored in the database securely. Users can log in using the registered credentials.
Alternative Flow	Invalid Input data If an input is invalid, the system will display error messages to get customers to enter the correct input and resubmit the form 1. Invalid E-mail • The system display "This email has already been taken. Please use a different email." 2. Incomplete form • User submitted a form containing unfilled field(s) • The system display "The form is not complete. Please fill in all the fields." 3. Weak password • The password does not meet the security criteria

	• System display “Password must be 8 characters long and includes uppercase, lowercase, numbers and special symbols (e.g. @, #, \$,!)” 4. System error or user error • System fails to process registration or user has connection issue • Account is not created • System displays “Registration failed, please try again.”
--	--

1.5.2.1.2 Login/Logout

Use Case	02
Use Case Name	Login/Logout
Actor	Customer/User
Description	This use case allow a registered customer to access the system's feature by entering valid login credentials and also end their session to ensure no access to their account.
Pre-condition	Login: • Customer has already registered • Customer is not currently logged in. • Login page is accessible Logout: • Customer is already logged in. • Login button is accessible
Basic Course Flow	Login 1. The user in navigates through the system to find and click the “Login” button which will lead to the login page. 2. The user enters and submits the valid credentials of their account (Email and password). 3. The system validates the input submitted. 4. If the validations are successful, the system redirects the user to the homepage or the previous page of the system the user was on Logout 1. User navigates to the “Logout” button. 2. User's session is terminated.
Post-Condition	Login: User is successfully logged into the system. A session is created and user can access more features. Logout: User is logged out of the system and directed to the homepage as a guest user. Active session is ended.
Alternative Flow	1. Forget password • User may click on “Forgot Password” • The system will load the password reset page that asks the user to input their email that is linked to their account so the system can send a link for user to reset and create a new password. 2. Incomplete required field • System will display an error message if either the email or password field is not filled in. 3. Invalid credentials • System display “Email or password entered does not match. Please enter the right credentials.”

1.5.2.1.3 View Item

Use Case	03
Use Case Name	View Item
Actor	Customer/User
Description	This use case allows the customer to browse the catalogue and view specific details of a grocery item, such as its name, price, product image, description, and stock availability.
Pre-condition	1. The user must be able to load the system. 2. Items must already exist in the system's database.
Basic Course Flow	1. Customer navigates through the product categories on the homepage. 2. Customer clicks on a specific product. 3. System retrieves the product information from the database. 4. System loads and displays the full item details to the customer.
Post-Condition	The customer successfully views the detailed information of the selected product.
Alternative Flow	Database/Connection Error 1. System fails to retrieve the item details from the database. 2. System displays an error message: "Unable to load product details. Please try again."

1.5.2.1.2 Customer

1.5.2.2.1 Search Item

Use Case	04
Use Case Name	Search Item
Actor	Customer
Description	This use case enables the customer to quickly locate specific grocery items by entering keywords into the search bar, rather than browsing manually through categories.
Pre-condition	1. The system is connected to the database. 2. The search bar is accessible on the interface.
Basic Course Flow	1. Customer clicks on the search bar. 2. Customer types in a keyword (e.g., "Milo", "chicken", "rice"). 3. Customer clicks the search icon or presses Enter. 4. System queries the database for items matching the keyword. 5. System displays a list of filtered search results to the customer.
Post-Condition	The customer views a specific list of products that match their search query.
Alternative Flow	No Results Found 1. Customer enters a keyword that does not match any items in the catalogue. 2. System displays a message: "No products found matching your search." 3. System prompts the customer to check their spelling or browse by category.

1.5.2.2.2 Manage Cart

Use Case	05
Use Case Name	Manage Cart
Actor	Customer
Description	This use case allows the customer to manage items in the shopping cart. Users can view the content of the cart, add products, update quantities of any product and remove products before proceeding to checkout.
Pre-condition	1. Customer must be logged into the system. 2. Products are available in the system.
Basic Course Flow	1. Customer searches for products. 2. Customer selects and adds the product to the cart. 3. Customer views the cart. 4. Customer updates product quantity or removes items if needed. 5. Customer selects the product they want to checkout. 6. System recalculates the total price. 7. Customer proceeds to checkout.
Post-Condition	1. Selected items are stored in the customer's cart. 2. Any updates in the cart are saved. 3. Items that have been checked out are removed from the cart.
Alternative Flow	Product in cart is out of stock • When system detected that a product is sold out, it will display a "Out of stock" message • Out of stock products in the cart are not available for check out; customers only have the option to remove the product from the cart. Invalid quantity input • Customer enters invalid quantity (e.g., zero, negative number or non-number characters). • System displays an error message. • Customer is prompted to enter a valid quantity. System Error • System fails to update cart information. • System displays an error message. • Customer may retry the action later.

1.5.2.2.3 Place Order

Use Case	06
Use Case Name	Place order
Actor	Customer
Description	This use case allows a logged-in customer to submit their selected cart items as a formal order by choosing a fulfilment method and confirming delivery or collection details.
Pre-condition	1. Customer is logged in. 2. Cart contains at least one item. 3. All cart items are in stock. 4. System is connected to the database.
Basic Course Flow	1. Customer proceeds to checkout from the cart page. 2. System displays order summary including items, quantities, and subtotal. 3. Customer selects fulfilment method: delivery or self-collection. 4. If delivery, customer enters or confirms delivery address. 5. Customer confirms the order. 6. System generates a unique Order ID and saves the order to the database with status "Order Confirmed". 7. System redirects customer to the payment page.
Post-Condition	A new order record is created in the database. Order status is set to "Order Confirmed". Customer is directed to make payment.
Alternative Flow	Out of stock detected at checkout: System displays an error indicating which item became unavailable. Customer is redirected to cart to remove or replace the item. System error: If the order fails to save, system displays "Order could not be placed. Please try again."

1.5.2.2.4 Make payment

Use Case	07
Use Case Name	Make Payment
Actor	Customer
Description	This use case allows the customer to complete their purchase by selecting and processing a payment method after placing an order.
Pre-condition	1. Customer has successfully placed an order. 2. Order ID exists in the database. 3. Payment gateway is accessible.
Basic Course Flow	1. System displays the payment page with order total and available methods: FPX, Debit/Credit Card, or COD. 2. Customer selects a payment method. 3. Customer enters required payment details (e.g., bank selection for FPX, card number for card payment). 4. Customer submits payment. 5. System processes the payment through the selected gateway. 6. System confirms successful transaction and updates order status to "Processing". 7. System displays a payment confirmation and order summary to the customer.
Post-Condition	Payment is recorded. Order status is updated to "Processing". Customer receives a confirmation notification.
Alternative Flow	Payment failure: Gateway returns a declined response. System displays "Payment unsuccessful. Please try a different method." Order status remains "Order Confirmed". COD selected: No gateway processing needed. System immediately updates order status to "Processing" and notes payment as pending on delivery.

1.5.2.2.4 Track Order

Use Case	07
Use Case Name	Track order
Actor	Customer
Description	This use case allows the customer to monitor the real-time status of their placed order from confirmation through to fulfilment.
Pre-condition	1. Customer is logged in. 2. At least one order has been placed by the customer.
Basic Course Flow	1. Customer navigates to "My Orders" section. 2. System retrieves and displays a list of the customer's orders. 3. Customer selects a specific order. 4. System displays the current order status and a progress timeline: Order Confirmed → Processing → Out for Delivery → Delivered. 5. System updates the displayed status automatically when the admin changes it.
Post-Condition	Customer views the latest order status. No changes are made to the order record.
Alternative Flow	No orders found: System displays "You have no orders yet." Network error: System displays "Unable to retrieve order status. Please try again later."

1.5.2.3 Admin

1.5.2.3.1 Login/Logout

Use Case	02
Use Case Name	Login/Logout
Actor	Admin
Description	This use cases allow the admin to access the system's administrative functions such as managing user accounts, products and orders.
Pre-condition	Login: • Admin is not currently logged in. • System is connected to the network. Logout: • Admin already logged in.
Basic Course Flow	Login 1. Admin navigates to the “Login” button. 2. System load the login page. 3. Admin enters its account's login credentials (email and password). 4. The system validates the input submitted. 5. If the validations are successful, the system redirects the admin to the homepage. Logout 1. Admin navigates to the “Logout” button. 2. Admin's session is terminated.
Post-Condition	Login: Admin is successfully logged into the system. A session is created and can access administrative features. Logout: Admin is logged out of the system and directed to the homepage as a guest user and cannot access admin's features. Active session is ended.
Alternative Flow	1. Forget password • Admin may click on “Forgot Password” • The system will load the password reset page that asks the admin to input their email that is linked to their account so the system can send a link for the admin to reset and create a new password. 2. Incomplete required field • System will display an error message if either the email or password field is not filled in. 3. Invalid credentials • System display “Email or password entered does not match. Please enter the right credentials.”

1.5.2.3.2 View item

Use Case	09
Use Case Name	View Item
Actor	Admin
Description	This use case allows the admin to view the full product catalogue and individual item details (such as name, category, price, stock quantity, and image) for monitoring and management purposes.
Pre-condition	1. Admin is logged into the system. 2. System is connected to the network and database.
Basic Course Flow	1. Admin navigates to the item management section. 2. System retrieves and displays the list of all items from the database. 3. Admin clicks on a specific item to view its full details. 4. System displays the detailed view of the selected item.
Post-Condition	Admin successfully views the detailed information of the selected item.
Alternative Flow	Database Connection Error 1. System fails to retrieve items from the database. 2. System displays an error message: "Unable to load catalogue at this time." 3. Admin may refresh the page to retry.

1.5.2.3.3 Add Item

Use Case	10
Use Case Name	Add Item
Actor	Admin
Description	This use case enables the admin to add new products to the system's catalogue by providing necessary details like name, category, price, stock quantity, and uploading a product image.
Pre-condition	1. Admin is logged into the system. 2. System is connected to the database.
Basic Course Flow	1. Admin navigates to the item management section and clicks "Add New Item". 2. System displays the item creation form. 3. Admin fills in item details (name, price, stock, category, description) and uploads a product image. 4. Admin submits the form. 5. System validates the inputs. 6. System saves the new item to the database. 7. System displays a success message.
Post-Condition	A new item is securely added to the database and becomes visible to customers in the catalogue.
Alternative Flow	Invalid Input Data 1. System detects missing required fields or invalid values (e.g., negative price or letters in a number field). 2. System displays an error message next to the respective fields. 3. Admin corrects the data and resubmits the form.

1.5.2.3.4 Delete Item

Use Case	11
Use Case Name	Delete Item
Actor	Admin
Description	This use case allows the admin to permanently remove a product from the system's catalogue when it is discontinued or no longer sold.
Pre-condition	1. Admin is logged into the system. 2. The specific item currently exists in the database.
Basic Course Flow	1. Admin navigates to the item management section and selects an item from the list. 2. Admin clicks the "Delete" button for that item. 3. System prompts for confirmation ("Are you sure you want to delete this item?"). 4. Admin confirms the deletion. 5. System removes the item record from the database. 6. System displays a successful deletion message.
Post-Condition	The item is permanently removed from the catalogue and database, and is no longer visible to customers.
Alternative Flow	Admin Cancels Deletion 1. At step 3, Admin clicks "Cancel" instead of confirming. 2. The system closes the prompt window. 3. The item remains untouched in the catalogue.

1.5.2.3.5 Update Item

Use Case	12
Use Case Name	Update Item
Actor	Admin
Description	This use case allows the admin to update existing item information in the catalogue of the system, such as item name, price, stock quantity, description, or product image.
Pre-condition	1. Is logged in as admin 2. Item has already been added in the catalogue 3. The system must be connected to the network and database.
Basic Course Flow	1. Admin navigates to the item management section. 2. System displays the list of existing items. 3. Admin selects the item to update. 4. System displays the current item details. 5. Admin modifies the required information. 6. Admin submits the updated item details. 7. System validates the updated information. 8. If validation is successful, the system saves the updated item information. 9. System displays a successful update message.
Post-Condition	Changes to an item is successfully updated and is visible to the customers
Alternative Flow	Invalid Input Data 1. System detects invalid or incomplete input data. 2. System displays error messages. 3. Admin is prompted to correct the information. Item Not Found 1. Selected item does not exist in the database. 2. System displays an error message. 3. Update process is cancelled. System Error 1. System fails to save updated item information. 2. System displays an error message. 3. Admin may retry the update later.

1.5.2.3.6 Search Item

Use Case	13
Use Case Name	Search Item
Actor	Admin
Description	This use case allows the admin to search for specific items in the catalogue using keywords, categories, or item IDs to quickly locate products for updating, viewing, or deletion.
Pre-condition	1. Admin is logged into the system. 2. Items exist in the catalogue.
Basic Course Flow	1. Admin enters a keyword or ID into the search bar. 2. Admin clicks the "Search" button or presses Enter. 3. System queries the database for matching item records. 4. System displays the filtered search results to the admin
Post-Condition	Admin views the specific list of items matching the search criteria, narrowing down the catalogue view.
Alternative Flow	No Items Found 1. Admin enters a search term that does not match any items in the database. 2. System displays a message: "No items match your search criteria." 3. Admin clears the search bar to view the full catalogue again.

1.5.2.3.7 Manage Order

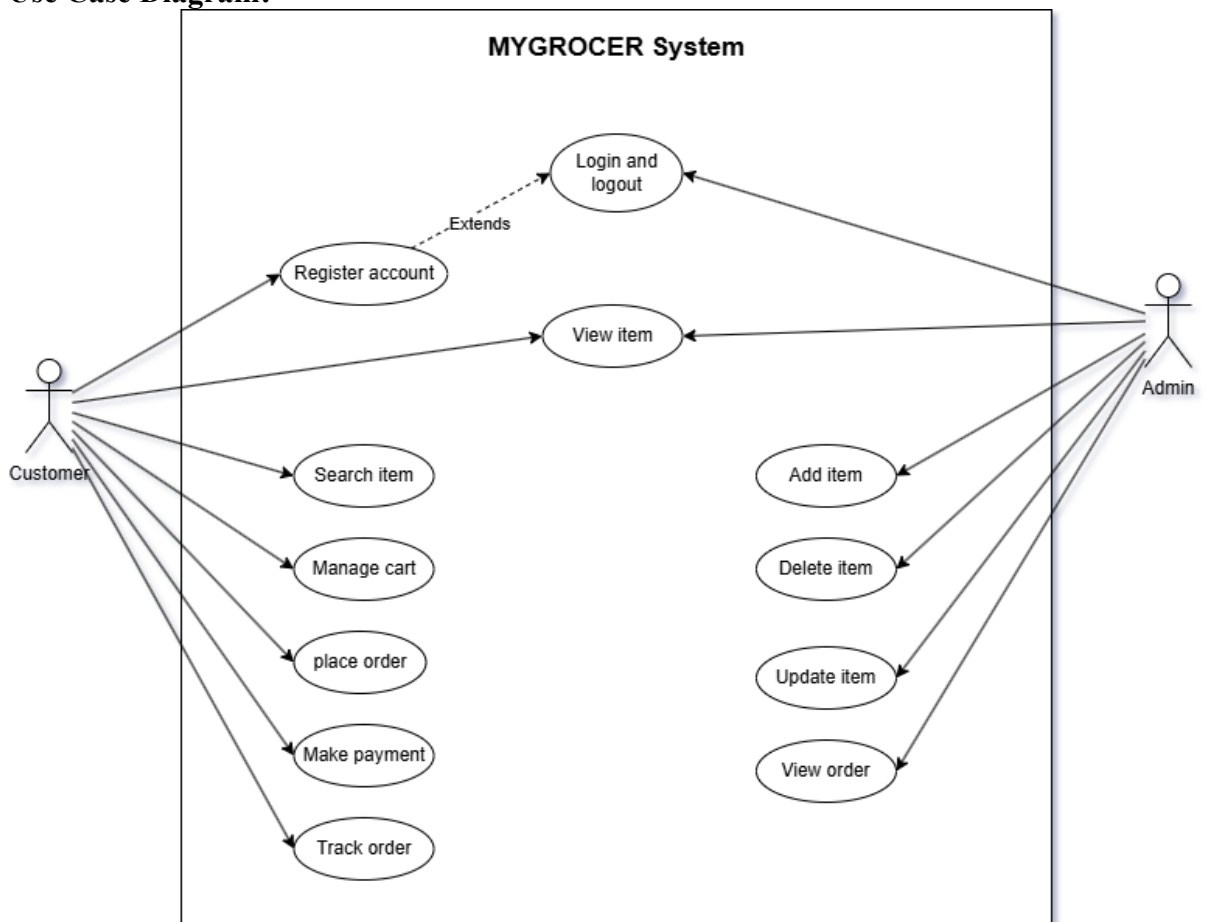
Use Case	14
Use Case Name	Manage Order
Actor	Admin
Description	This use case allows the admin to view all customer orders and update their statuses to reflect the current fulfilment progress, which automatically notifies the relevant customer.
Pre-condition	1. Admin is logged in. 2. Orders exist in the system database..
Basic Course Flow	1. Admin navigates to the Order Management section. 2. System displays a list of all orders with details: Order ID, customer name, items, total, fulfilment method, and current status. 3. Admin selects a specific order to view its full details. 4. Admin updates the order status (e.g., from "Processing" to "Out for Delivery"). 5. System saves the updated status to the database. 6. Observer pattern automatically triggers a notification to the subscribed customer. 7. System displays a success message to the admin.
Post-Condition	Order status is updated in the database. Customer is notified of the new status in real time.
Alternative Flow	No orders found: System displays "No orders available." System error on update: System displays an error message and retains the previous status.

2.0 System Overview

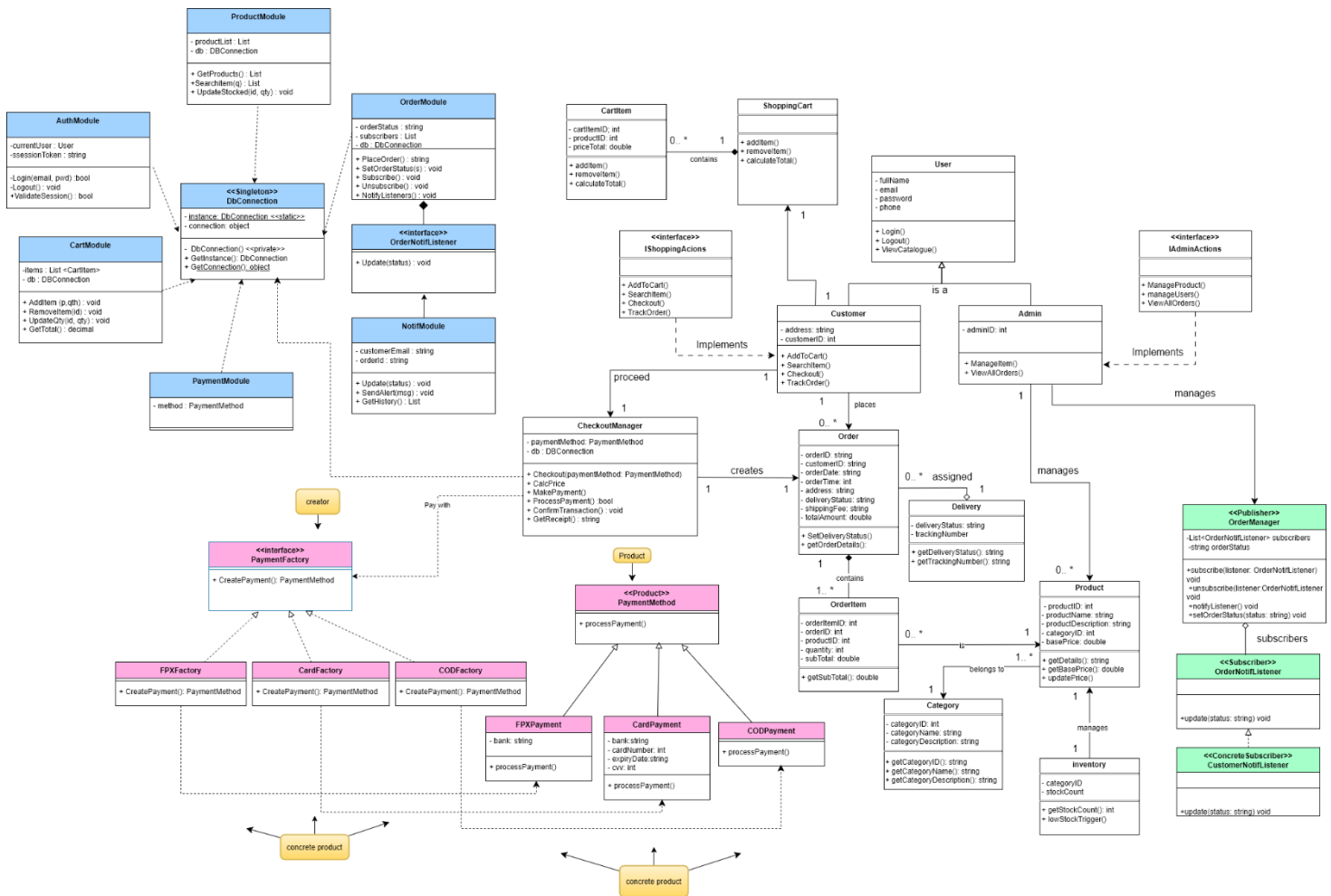
2.1 Generic Use Case

Actor	Use Case
Customer	1. Register Account 2. Login and Logout 3. View Item 4. Search Item 5. Manage Cart 6. Place Order 7. Make Payment 8. Track Order
Admin	1. Login and Logout 2. View Item 3. Add item 4. Delete item 5. Update item 6. Manage order

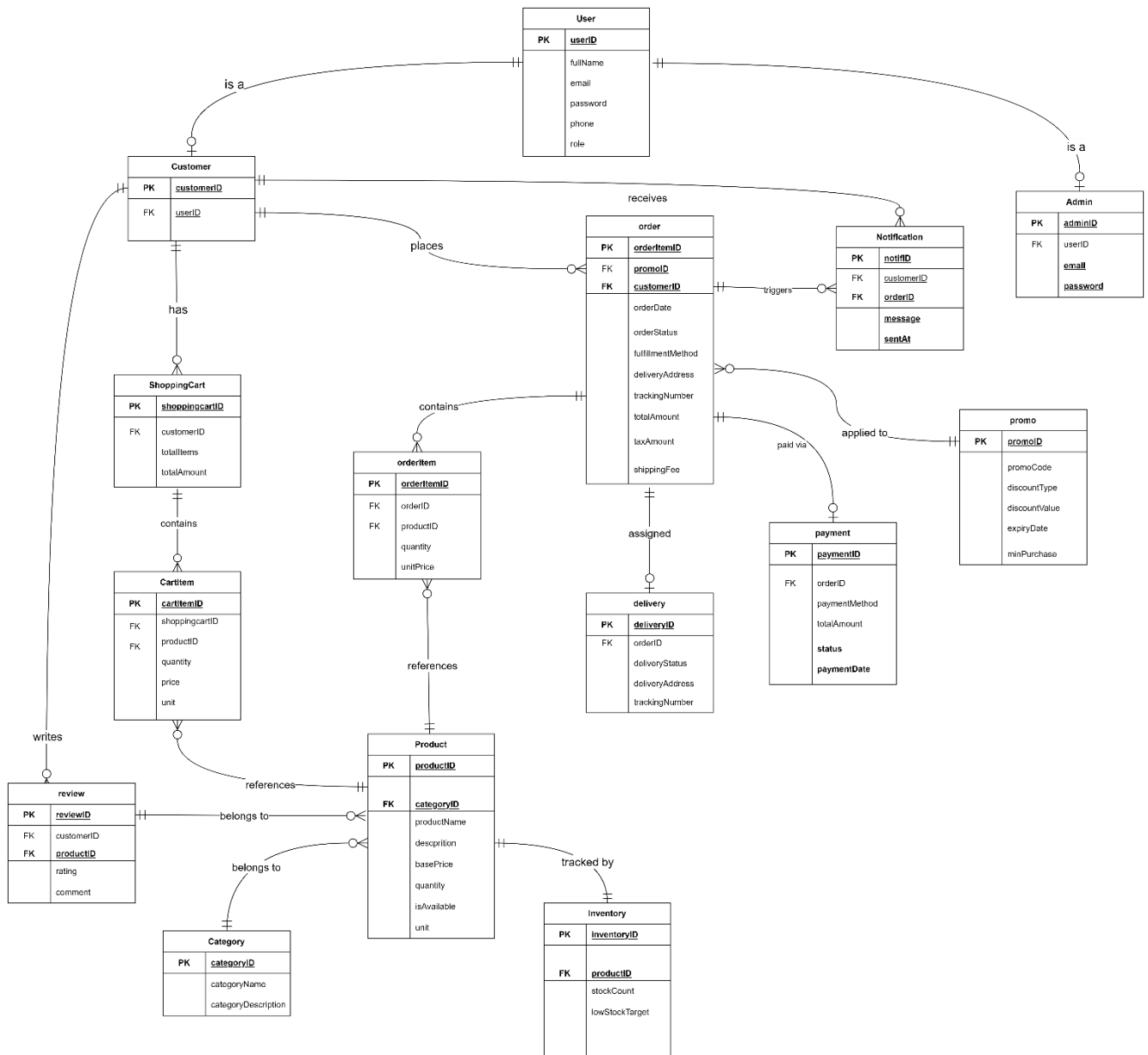
Use Case Diagram:



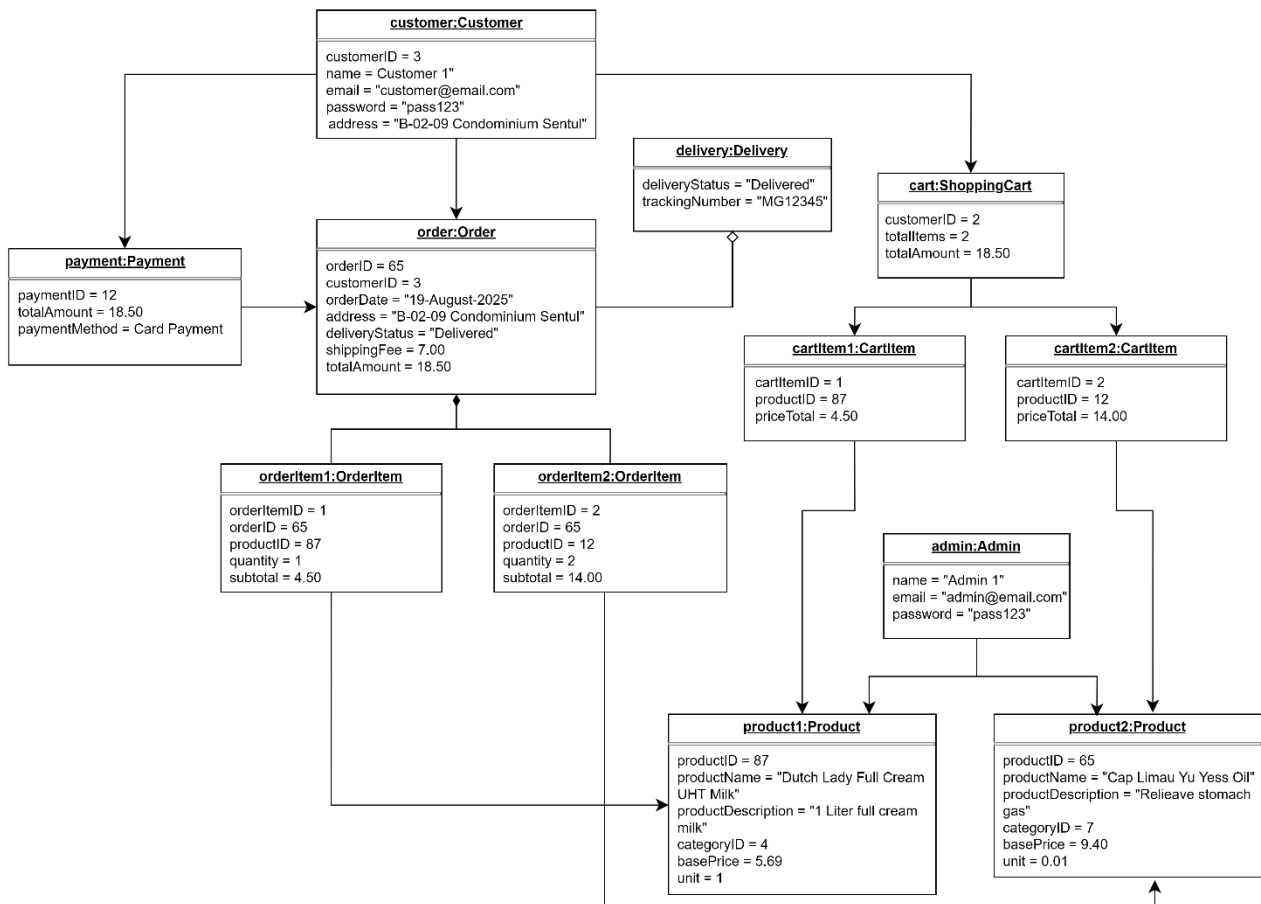
2.2 Class Diagrams



2.3 ERD



2.4 Object Diagrams



3.0 Requirements

3.1 Software Design Concepts

3.1.1 Abstraction

Abstraction is a software design concept that involves hiding complex implementation details and exposing only the essential features or behaviours that are relevant to the user or other parts of the system. In MYGROCER, abstraction is applied across multiple layers to simplify system interaction and reduce implementation complexity.

How it is applied in MYGROCER:

The PaymentMethod abstract class is the most prominent example of abstraction in this system. Rather than exposing the internal logic of how FPX, card, or COD transactions are processed, the system presents a single unified method ProcessPayment() to the checkout module. The checkout component does not need to know how each payment gateway works internally it only calls ProcessPayment() and the correct subclass handles the rest.

Similarly, the Product abstract concept hides the differences between product categories (Fresh Produce, Packaged Goods, Beverages, Household). The system treats all products through a common interface without exposing category-specific instantiation logic to the client.

At the user management level, the User abstract class hides role-specific behaviour. The AuthenticationSystem client calls Login() and ViewCatalogue() on any User object without knowing whether it is dealing with a Customer or an Admin.

Sample code (C #) before abstraction (all payment logic hardcoded directly in Checkout);

```
// Problem: Checkout must know HOW every payment works internally.
// Adding a new method means rewriting Checkout itself.

class Checkout
{
    public void ProcessPayment(string type, string bankCode, string cardNumber)
    {
        if (type == "FPX")
        {
            // FPX logic exposed directly here
            Console.WriteLine("Connecting to FPX gateway...");
            Console.WriteLine("Bank code: " + bankCode);
            Console.WriteLine("FPX Payment Processed.");
        }
        else if (type == "CARD")
        {
            // Card logic exposed directly here
            Console.WriteLine("Connecting to card network...");
            Console.WriteLine("Card number: " + cardNumber);
        }
    }
}
```

```

        Console.WriteLine("Card Payment Processed.");
    }
    else if (type == "COD")
    {
        // COD logic exposed directly here
        Console.WriteLine("No gateway needed.");
        Console.WriteLine("Cash on Delivery confirmed.");
    }
    // To add GrabPay, a developer must edit THIS class and add another else-if
}
}

```

Problem: The Checkout class is exposed to every internal detail of every payment method. There is no shared contract nothing hides the complexity from the caller.

Sample code (C#) after abstraction:

```

// AFTER Abstraction: Complex implementation hidden behind a common abstract
// contract
// Checkout only calls ProcessPayment() — it never knows HOW each method works
// internally

// Abstract class defines the contract — hides implementation details from client code
abstract class PaymentMethod
{
    public abstract void ProcessPayment();
}

// FPX implementation hidden inside this class — Checkout never sees this logic
class FPXPayment : PaymentMethod
{
    private string bankCode = "MBB0227"; // Internal detail — hidden from outside

    public override void ProcessPayment()
    {
        Console.WriteLine("Connecting to FPX gateway...");
        Console.WriteLine("Bank code: " + bankCode);
        Console.WriteLine("FPX Payment Processed Successfully.");
    }
}

// Card implementation hidden inside this class
class CardPayment : PaymentMethod
{
    private string cardNetwork = "VISA"; // Internal detail — hidden from outside

    public override void ProcessPayment()
    {
        Console.WriteLine("Connecting to card network: " + cardNetwork);
        Console.WriteLine("Card Payment Processed Successfully.");
    }
}

```

```

}

// COD implementation hidden inside this class
class CODPayment : PaymentMethod
{
    public override void ProcessPayment()
    {
        Console.WriteLine("No gateway required.");
        Console.WriteLine("Cash on Delivery confirmed. Payment pending on arrival.");
    }
}

// Checkout only depends on the abstraction — not on any concrete class
class Checkout
{
    public void ProcessPayment(PaymentMethod method)
    {
        // Does NOT know whether this is FPX, Card, or COD
        // Just calls the unified contract — complexity is fully hidden
        method.ProcessPayment();
    }
}

// Client code
class Program
{
    static void Main(string[] args)
    {
        Checkout checkout = new Checkout();

        // Swap payment types without changing Checkout at all
        PaymentMethod payment = new FPXPayment();
        checkout.ProcessPayment(payment);

        payment = new CardPayment();
        checkout.ProcessPayment(payment);

        payment = new CODPayment();
        checkout.ProcessPayment(payment);
    }
}

```

Expected Output:

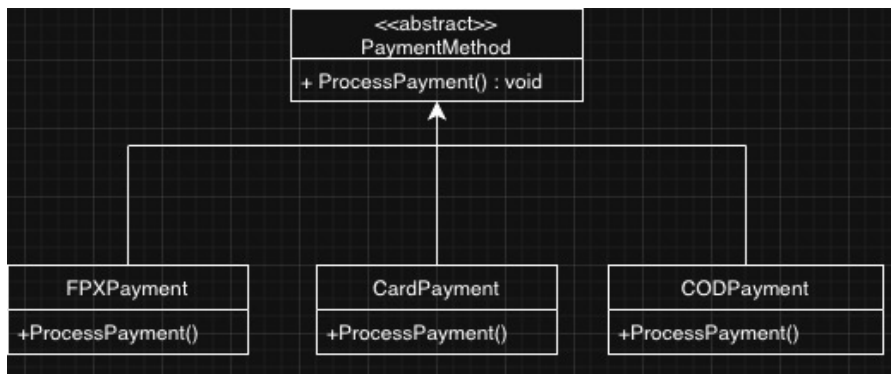
```

Connecting to FPX gateway...
Bank code: MBB0227
FPX Payment Processed Successfully.
Connecting to card network: VISA
Card Payment Processed Successfully.
No gateway required.
Cash on Delivery confirmed. Payment pending on arrival.

```

What changed: `Checkout.ProcessPayment()` no longer contains any `if/else` logic. It receives any `PaymentMethod` object and calls the same single method — the internal details of each payment type are completely hidden inside their own classes. Adding `GrabPay` in the future only requires a new `GrabPayPayment : PaymentMethod` class with no changes to `Checkout` at all.

Sample Class Diagram (Abstraction via PaymentMethod):



Benefits:

- I. Simplifies the interface exposed to client code, reducing the cognitive load on developers.
- II. Allows the system to be extended with new product types or payment methods without modifying existing consumer code.
- III. Promotes separation of concerns by keeping implementation details hidden within subclasses.

3.1.2 Encapsulation

In the User class, sensitive attributes such as password, email, and phone are declared as private. These cannot be accessed or modified directly from outside the class. Instead, controlled access is provided through public methods such as Login() and Logout(). This ensures that password data, for example, is never exposed to other system components.

In the DbConnection class (Singleton Pattern), the database connection instance is kept private static, meaning no external component can directly instantiate or tamper with the connection object. The only way to access it is through the public static method getInstance(), which enforces the Singleton guarantee.

In the OrderManager class (Observer Pattern), the orderStatus attribute is private. It can only be changed via the public method setOrderStatus(status), which also triggers notifyListeners(). This controlled mutation ensures that every status change is automatically broadcast to subscribers, maintaining data consistency.

Before Encapsulation:

```
// All fields are public, any class can read or modify them directly
// Problem: Password, email, and order status can be changed from anywhere
// with no validation or controlled access.

class User
{
    public string fullName;
    public string email;
    public string password;    // Anyone can read or overwrite this directly
    public string phone;
}

class OrderManager
{
    public string orderStatus;    // Can be changed without triggering any
    notification
    public List<string> subscribers = new List<string>();
}

class DbConnection
{
    public static DbConnection instance; // External code can assign a fake
    instance
    public object connection;    // Anyone can swap the connection object
}
```

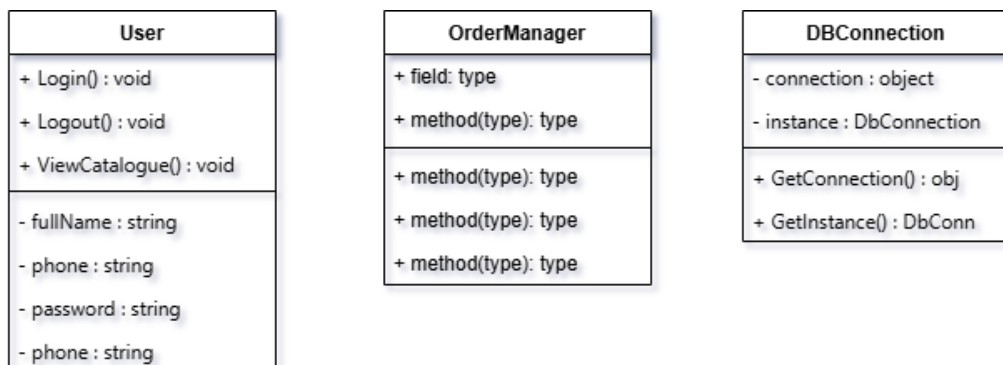
```
// Example of the danger this creates:

class SomeOtherClass
{
    public void Tamper()
    {
        User u = new User();
        u.password = "hacked"; // No validation – accepted silently
        OrderManager om = new OrderManager();

        om.orderStatus = "Delivered";
        // Status changed but NO notification sent
        // Customer never finds out their order "arrived"
    }
}
```

Problem: Without private fields and controlled methods, any part of the system can corrupt data silently. A status change on OrderManager skips NotifyListeners() entirely

Class Diagram:



private fields — no direct access

After Encapsulation Sample Code(C#):

```
// Encapsulation in User class
class User
{
    private string fullName;
    private string email;
    private string password; // Cannot be accessed directly from outside
    private string phone;
    public virtual void Login()
```

```

    {
        Console.WriteLine("User logged in.");
    }
    public virtual void Logout()

    {
        Console.WriteLine("User logged out.");
    }
}

```

```

// Encapsulation in OrderManager
class OrderManager
{
    private string orderStatus; // Hidden from external modification
    public void SetOrderStatus(string status) {
        orderStatus = status;
        NotifyListeners(); // Controlled change triggers notification
    }
    public void NotifyListeners()
    {
        Console.WriteLine("Notifying observers of status: " + orderStatus);
    }
}

```

Benefits:

- I. Protects sensitive data such as passwords and database connections from unintended external modification.
- II. Enforces data integrity by ensuring state changes happen only through defined methods.
- III. Makes the system easier to maintain because the internal implementation of a class can be changed without affecting other components that depend on it.

3.1.3 Modularity

In the MYGROCER system, modularity is applied by separating the system into several functional modules such as:

- User Management Module
- Product Management Module
- Shopping Cart Module
- Payment Module
- Order Management Module
- Notification Module

Every module manages its own responsibilities without interfering directly with the internal implementation of other modules. For example, the Payment Module is responsible only for processing payments, while the Product Management Module handles stock updates and product availability. This reduces system complexity and make it maintainable and extensible.

The implementation modularity also supports teamwork during because it all allow different developers to work on separate modules simultaneously without affecting other modules. Moreover, modularity improves scalability in MYGROCER. New features such as additional payment gateways, loyalty programs, or delivery services can be added by modifying or extending only the related module without changing the entire system. This reduces the risk of introducing errors into unrelated components.

Using design patterns also strengthen the modularity of this system. The Factory Method Pattern modularizes payment creation logic, the Singleton Pattern centralizes database connection management, and the Observer Pattern separates notification handling from order processing logic. These implementations ensure that the system remains flexible, maintainable, and easier to enhance in the future.

3.1.4 Functional Independence

Functional independence is a software design concept that refers to the degree to which a module or component performs a single, well-defined function with minimal dependency on other modules. It is measured through two metrics: cohesion (how strongly related the responsibilities within a module are) and coupling (how dependent modules are on each other). High cohesion and low coupling are the goals of functional independence.

Sample Code (C#) before Functional Independence:

```
// BEFORE Functional Independence: One massive class handles everything
// Problem: High coupling, low cohesion. A bug in payment breaks order
// tracking.
//          Cannot test modules independently. One developer blocks all
// others.

class GrocerSystem
{
    // Authentication responsibility
    private string currentUserEmail;
    private string currentUserPassword;

    public bool Login(string email, string password)
    {
        // Validate credentials
        currentUserEmail = email;
        Console.WriteLine("User logged in: " + email);
        return true;
    }

    // Product responsibility mixed in
    private List<string> productList = new List<string>();

    public List<string> GetProducts()
    {
        Console.WriteLine("Fetching products from DB...");
        return productList;
    }

    public void UpdateStock(int id, int qty)
    {
        Console.WriteLine($"Stock updated: item {id} → {qty} units");
    }

    // Cart responsibility mixed in
    private List<string> cartItems = new List<string>();
}
```

```

public void AddToCart(string product)
{
    cartItems.Add(product);
}

public decimal GetTotal()
{
    return cartItems.Count * 10.00m; // Oversimplified – all mixed
together
}

// Payment responsibility mixed in
public bool ProcessPayment(string method)
{
    Console.WriteLine("Processing payment via: " + method);
    return true;
}

// Order + Notification responsibility mixed in
public void PlaceOrder()
{
    Console.WriteLine("Order placed.");
    Console.WriteLine("Sending notification..."); // Notification
tightly coupled here
    Console.WriteLine("Saving to database..."); // DB call also
lives here
}
}

// Changing payment logic risks breaking Login().
// Cannot unit-test CartModule without also running authentication.

```

Problem: Every concern lives in one class. There is no independence a change to any one feature risks breaking all the others.

Sample Code (C#) after Functional Independence :

```

// AFTER Functional Independence: Each module handles exactly one concern
// High cohesion within each module, low coupling between them

// ——— Auth Module
_____

// Only responsibility: login, logout, session validation
class AuthModule
{
    private string currentUser;
    private string sessionToken;

```

```

public bool Login(string email, string pwd)
{
    currentUser = email;
    sessionToken = "TOKEN_" + email;
    Console.WriteLine("AuthModule: User logged in → " + email);
    return true;
}

public void Logout()
{
    currentUser = null;
    sessionToken = null;
    Console.WriteLine("AuthModule: Session ended.");
}

public bool ValidateSession()
{
    return sessionToken != null;
}
}

// ——— Product Module


---


// Only responsibility: product catalogue and stock management
class ProductModule
{
    private List<string> productList = new List<string>();
    private DbConnection db = DbConnection.GetInstance();

    public List<string> GetProducts()
    {
        Console.WriteLine("ProductModule: Fetching product catalogue...");
        return productList;
    }

    public void SearchItem(string query)
    {
        Console.WriteLine("ProductModule: Searching for → " + query);
    }

    public void UpdateStocked(int id, int qty)
    {
        Console.WriteLine($"ProductModule: Stock updated → item {id}, qty {qty}");
    }
}

```

```
// ——— Cart Module


---


// Only responsibility: managing cart state (add, update, remove, total)
// No knowledge of payment or order logic
class CartModule
{
    private List<string> items = new List<string>();
    private DbConnection db = DbConnection.GetInstance();

    public void AddItem(string product, int qty)
    {
        items.Add(product);
        Console.WriteLine($"CartModule: Added {product} × {qty}");
    }

    public void RemoveItem(int id)
    {
        Console.WriteLine("CartModule: Item removed → id " + id);
    }

    public void UpdateQty(int id, int qty)
    {
        Console.WriteLine($"CartModule: Quantity updated → item {id}, qty {qty}");
    }

    public decimal GetTotal()
    {
        Console.WriteLine("CartModule: Calculating total...");
        return items.Count * 10.00m;
    }
}

// ——— Payment Module


---


// Only responsibility: processing payment via selected method
// Depends only on abstract PaymentMethod — no cart or order logic inside
class PaymentModule
{
    private PaymentMethod method;
    private DbConnection db = DbConnection.GetInstance();

    public void SelectMethod(string type)
    {
        if (type == "FPX") method = new FPXPayment();
        else if (type == "CARD") method = new CardPayment();
        else method = new CODPayment();
        Console.WriteLine("PaymentModule: Method selected → " + type);
    }
}
```

```

public bool ProcessPayment()
{
    method.ProcessPayment();
    return true;
}

public void ConfirmTransaction()
{
    Console.WriteLine("PaymentModule: Transaction confirmed.");
}
}

// ——— Order Module

```

```

// Only responsibility: order lifecycle and status updates
// Notifies observers independently — does not handle payment or cart
class OrderModule
{
    private string orderStatus;
    private List<OrderNotifListener> subscribers = new List<OrderNotifListener>();
    private DbConnection db = DbConnection.GetInstance();

    public string PlaceOrder()
    {
        orderStatus = "Order Confirmed";
        Console.WriteLine("OrderModule: Order placed → " + orderStatus);
        return "ORD-" + DateTime.Now.Ticks;
    }

    public void SetOrderStatus(string status)
    {
        orderStatus = status;
        NotifyListeners();
    }

    public void Subscribe(OrderNotifListener listener)
    {
        subscribers.Add(listener);
    }

    public void Unsubscribe(OrderNotifListener listener)
    {
        subscribers.Remove(listener);
    }

    public void NotifyListeners()
    {
        foreach (var s in subscribers)

```

```

        s.Update(orderStatus);
    }
}

// ——— Notification Module


---


// Only responsibility: delivering status alerts to customers
// Operates independently — OrderModule does not know how alerts are sent
class NotifModule : OrderNotifListener
{
    private string customerEmail;
    private string orderId;

    public NotifModule(string email, string orderId)
    {
        this.customerEmail = email;
        this.orderId = orderId;
    }

    public void Update(string status)
    {
        Console.WriteLine($"NotifModule: Alert sent to {customerEmail} " +
            $"→ Order {orderId} is now '{status}'");
    }

    public void SendAlert(string msg)
    {
        Console.WriteLine("NotifModule: " + msg);
    }
}

// ——— Program: modules working together without interfering


---


class Program
{
    static void Main(string[] args)
    {
        // Each module is independent — changing one does not affect the others
        AuthModule auth = new AuthModule();
        ProductModule products = new ProductModule();
        CartModule cart = new CartModule();
        PaymentModule payment = new PaymentModule();
        OrderModule order = new OrderModule();
        NotifModule notif = new NotifModule\("customer@email.com", "ORD-001"\);

        order.Subscribe(notif);    // NotifModule plugs into OrderModule independently

        auth.Login\("customer@email.com", "pass123"\);
        products.GetProducts();
    }
}

```

```

    cart.AddItem("Fresh Milk 1L", 2);
    cart.GetTotal();
    payment.SelectMethod("FPX");
    payment.ProcessPayment();
    order.PlaceOrder();
    order.SetOrderStatus("Processing");
    order.SetOrderStatus("Out for Delivery");
    order.SetOrderStatus("Delivered");
    order.Unsubscribe(notif);
  }
}

```

Expected Output:

```

AuthModule: User logged in → customer@email.com
ProductModule: Fetching product catalogue...
CartModule: Added Fresh Milk 1L × 2
CartModule: Calculating total...
PaymentModule: Method selected → FPX
Processing FPX Payment...
OrderModule: Order placed → Order Confirmed
NotifModule: Alert sent to customer@email.com → Order ORD-001 is now 'Processing'
NotifModule: Alert sent to customer@email.com → Order ORD-001 is now 'Out for Delivery'
NotifModule: Alert sent to customer@email.com → Order ORD-001 is now 'Delivered'

```

How functional independence is actually applied in MYGROCER:

MYGROCER is structured around independent functional modules, each responsible for a specific concern:

Module	Responsibility	Independence
AuthenticationModule	Handles login, logout, session management	No dependency on payment or cart
ProductModule	Manages product catalogue, categories, stock	Independent of user or payment logic
CartModule	Manages adding, updating, removing cart items	No direct dependency on payment processing
PaymentModule	Processes payment using selected method	Depends only on abstract PaymentMethod
OrderModule	Manages order lifecycle and status updates	Notifies observers independently via Observer pattern
NotificationModule	Sends order status updates to customers/staff	Operates independently via OrderNotifListener
DConnection	Manages database connectivity (Singleton)	Isolated single instance, no business logic

This modular design means that a change in the PaymentModule (e.g., adding GrabPay) does not require any changes to the CartModule or OrderModule. Each module is cohesive it contains only what it needs and loosely coupled to others.

Benefits:

- I. Reduces the risk of cascading failures: a bug in the PaymentModule does not break order tracking.
- II. Simplifies testing because each module can be unit-tested independently.
- III. Promotes reusability for example, the NotificationModule could be reused for low-stock alerts to the admin without modification.
- IV. Accelerates team development since different members can work on different modules simultaneously without conflict.

3.1.5 Refinement

Refinement is a top-down software design strategy where a high-level, abstract concept is progressively elaborated into detailed, concrete components. It involves moving from a macroscopic view of the system down to the microscopic implementation details, ensuring that every low-level function serves the high-level requirement.

How it is applied in MYGROCER:

In MYGROCER, refinement is applied to the concept of Payment Processing. Initially, the system simply requires a "Payment Method". Through refinement, this abstract requirement is broken down into a defined contract (the `PaymentMethod` abstract class), and then further refined into specific, actionable implementations (`FPXPayment`, `CardPayment`, `CODPayment`). Each refined step adds specific logic, such as how FPX connects to banking APIs compared to how COD updates a delivery status.

Sample Code (C#):

```
// High-level abstraction
abstract class PaymentMethod
{
    public abstract void ProcessPayment();
}

// Refined into concrete, detailed implementation
class FPXPayment : PaymentMethod
{
    private string bankCode;

    public void SelectBank(string code)
    {
        bankCode = code;
    }

    // Detailed implementation of the abstract concept
    public override void ProcessPayment()
    {
        Console.WriteLine($"Connecting to FPX gateway for bank {bankCode}...");
        Console.WriteLine("FPX Payment Processed Successfully.");
    }
}
```

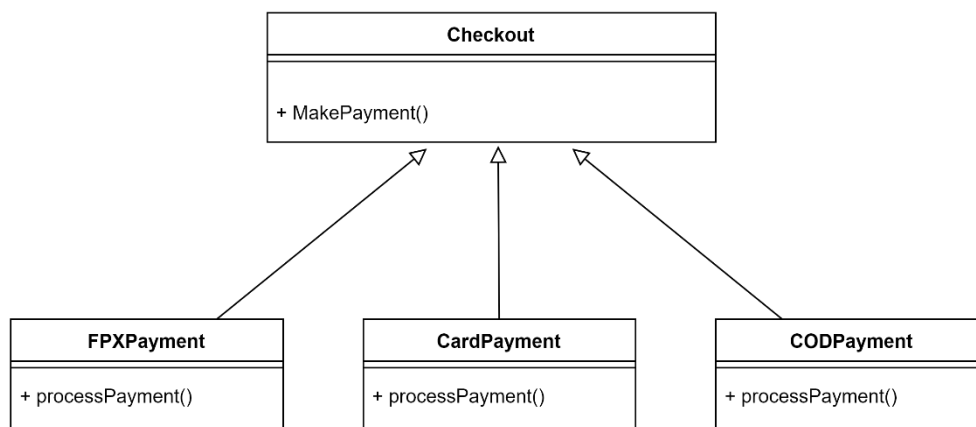
Benefits:

- **Manages Complexity:** Allows developers to focus on the overall system design before getting bogged down in intricate coding details.
- **Traceability:** Ensures that every specific piece of code (like the FPX API connection) maps directly back to a high-level system requirement (checkout processing).

3.1.6 Refactoring

In the MYGROCER system, refactoring is applied throughout the development process to ensure that the source code remains clean, modular, and maintainable. As the system grows with multiple modules such as payment processing, shopping cart management, inventory tracking, and order notifications, refactoring helps prevent the codebase from becoming overly complex and difficult to manage.

Before refactoring



```
class Checkout
{
    public void MakePayment(string type)
    {
        if(type == "FPX")
        {
            FPXPayment payment = new FPXPayment();
            payment.ProcessPayment();
        }

        else if(type == "CARD")
        {
            CardPayment payment = new CardPayment();
            payment.ProcessPayment();
        }

        else if(type == "COD")
        {
            CODPayment payment = new CODPayment();
            payment.ProcessPayment();
        }
    }
}
```

Problems in this coding:

-
- High coupling between Checkout and payment classes
- Difficult to add new payment methods
- Violates Open/Closed Principle (OCP)
- Checkout class has too many responsibilities
- Harder to maintain and test

After refactoring

```
// Product
abstract class PaymentMethod
{
    public abstract void ProcessPayment();
}

// Concrete Products
class FPXPayment : PaymentMethod
{
    public override void ProcessPayment()
    {
        Console.WriteLine("Processing FPX Payment...");
    }
}
class CardPayment : PaymentMethod
{
    public override void ProcessPayment()
    {
        Console.WriteLine("Processing Card Payment...");
    }
}
class CODPayment : PaymentMethod
{
    public override void ProcessPayment()
    {
        Console.WriteLine("Processing Cash On Delivery...");
    }
}

// Creator
abstract class PaymentFactory
{
    public abstract PaymentMethod CreatePayment();
}

// Concrete Creators
class FPXFactory : PaymentFactory
{
    public override PaymentMethod CreatePayment()
    {
        return new FPXPayment();
    }
}
```

```

class CardFactory : PaymentFactory
{
    public override PaymentMethod CreatePayment()
    {
        return new CardPayment();
    }
}

class CODFactory : PaymentFactory
{
    public override PaymentMethod CreatePayment()
    {
        return new CODPayment();
    }
}

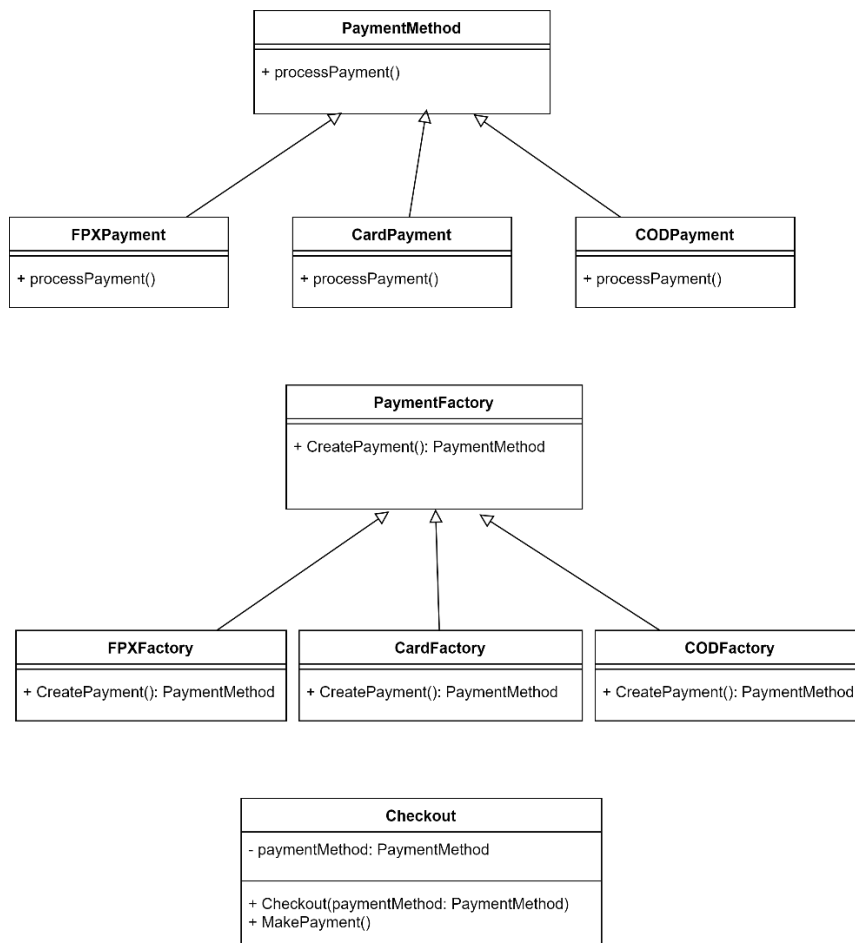
class Checkout
{
    private PaymentFactory factory;

    public Checkout(PaymentFactory factory)
    {
        this.factory = factory;
    }

    public void CompletePayment()
    {
        PaymentMethod payment = factory.CreatePayment();
        payment.ProcessPayment();

        Console.WriteLine("Payment completed successfully.");
    }
}

```



After refactoring, the system applies the Factory Method Pattern and Dependency Inversion Principle by separating payment creation into dedicated factory classes and abstractions. This improves scalability, flexibility, and code organization while making the system easier to maintain and extend in the future.

3.2.7 Architecture

Software architecture refers to the fundamental structure of a system, defining its main components, their relationships, and the principles guiding its design. It provides the overall blueprint for how the application operates and scales.

How it is applied in MYGROCER:

MYGROCER employs a layered, modular architecture that separates concerns into distinct tiers. It separates the Presentation Layer (the User Interface for browsing and checkout) from the Business Logic Layer (handling cart calculations, pricing rules, and order management) and the Data Access Layer (database interactions managed via the Singleton pattern). This strongly aligns with the Model-View-Controller (MVC) architectural style.

Sample Code (C#):

```
// Demonstrating Layered Architecture Concept

// 1. Data Access Layer (Model)
class DatabaseLayer
{
    public void SaveOrderToDB(string orderDetails) {
        Console.WriteLine("Executing SQL: INSERT INTO Orders...");
    }
}

// 2. Business Logic Layer (Controller/Service)
class OrderService
{
    private DatabaseLayer db = new DatabaseLayer();

    public void ProcessNewOrder(string cartData) {
        Console.WriteLine("Applying pricing rules and taxes to cart...");
        db.SaveOrderToDB("Processed " + cartData);
    }
}

// 3. Presentation Layer (View/UI boundary)
class CustomerUI
{
    private OrderService orderService = new OrderService();

    public void ClickCheckout() {
        Console.WriteLine("User clicked checkout.");
        orderService.ProcessNewOrder("CartContents");
    }
}
```

Benefits:

- Scalability: Different layers can be scaled independently. For example, the database layer can be upgraded without having to rewrite the customer interface.
- Maintainability: Bugs are easier to locate and fix because they are isolated within specific architectural layers, reducing the risk of breaking unrelated parts of the system.

3.1.8 Pattern

A software design pattern is a general, reusable solution to a commonly occurring problem within a given context in software design. It is not a finished piece of code, but a formalized template or best practice for how to solve a recurring architectural challenge.

How it is applied in MYGROCER:

MYGROCER utilizes three specific design patterns to solve core system challenges robustly:

1. **Singleton Pattern:** Ensures only one instance of the database connection exists to prevent resource exhaustion.
2. **Factory Pattern:** Dynamically instantiates different payment and product objects without exposing the creation logic to the client.
3. **Observer Pattern:** Handles decoupled, real-time order status notifications to both customers and admin systems. *(Note: Detailed implementations of these patterns are documented in Section 4.0).*

Sample Code (C#):

```
// A quick conceptual look at how patterns are utilized in the main flow
class CheckoutSystem
{
    public void CompletePurchase(string paymentType)
    {
        // Using Factory Pattern to get the right payment object
        PaymentFactory factory = new PaymentFactory();
        PaymentMethod payment = factory.CreatePayment(paymentType);

        payment.ProcessPayment();

        // Using Singleton Pattern to save the result
        DbConnection db = DbConnection.GetInstance();
        db.GetConnection(); // Save transaction

        // Observer Pattern triggers notifications in the background
    }
}
```

Benefits:

- **Proven Reliability:** Uses tested, industry-standard paradigms that prevent subtle issues which could cause major system failures later.
- **Improved Communication:** Standardizes the code language. When a developer sees a Factory or Singleton class, they immediately understand its purpose and behavior without reading every line of code.

3.2 Design Principles

3.2.1 Single Responsibility Principle (SRP)

The Single Responsibility Principle states that a class should have only one reason to change, meaning each class should be responsible for only one specific part of the system's functionality. When a class handles too many responsibilities, a change in one area risks breaking unrelated functionality.

How it is applied in MYGROCER:

Each class in MYGROCER is designed to handle a single, well-defined responsibility:

Class	Single Responsibility
User	Manages user identity and authentication behaviour.
Customer	Handles customer-specific shopping actions (search item, place order).
Admin	Handles administrative actions (manage item, view orders).
DbConnection	Manages database connection lifecycle only.
PaymentMethod	Defines the contract for payment processing.
FPXPayment / CardPayment/CODPayment	Each processes one specific payment type only.
OrderManager	Manages order status and notifies observers.
CustomerNotifListener	Handles customer-facing order status notifications only.
CartModule	Manages cart state: add, update, remove items.

Without SRP, a single Checkout class might handle payment processing, sending notifications, updating inventory, and saving order records giving it many reasons to change. In MYGROCER, these responsibilities are separated:

- PaymentFactory handles payment object creation.
- OrderManager handles order state.
- CustomerNotifListener handles notification delivery.
- DbConnection handles data persistence

Sample Code (C#) Before SRP:

```
// BEFORE SRP: One "God class" has too many reasons to change
// Problem: If payment logic changes, notification logic changes, or DB schema changes —
//         all require editing this one class, risking unintended breakage.

class Checkout
{
    public void CompleteOrder(string paymentType, string customerEmail)
    {
        // Responsibility 1: Payment processing
        if (paymentType == "FPX")
        {
            Console.WriteLine("Processing FPX Payment...");
        }
        else if (paymentType == "CARD")
        {
            Console.WriteLine("Processing Card Payment...");
        }
    }
}
```

```

else if (paymentType == "COD")
{
    Console.WriteLine("Processing COD...");
}

// Responsibility 2: Sending customer notification
Console.WriteLine("Sending email to: " + customerEmail);
Console.WriteLine("Subject: Your order is confirmed!");

// Responsibility 3: Updating inventory
Console.WriteLine("Deducting stock from inventory...");

// Responsibility 4: Saving order to database
Console.WriteLine("INSERT INTO Orders VALUES (...");

// Responsibility 5: Managing cart state
Console.WriteLine("Clearing cart for user: " + customerEmail);
}
}
// This class has 5 reasons to change.
// Any one change (e.g. adding GrabPay) risks accidentally breaking email delivery.

```

Problem: The Checkout class violates SRP by owning payment, notification, inventory, database, and cart logic all at once.

Sample Code (C#) After SRP:

```

// SRP: DbConnection only manages the database connection - nothing else
class DbConnection
{
    private static DbConnection instance;
    private object connection;

    private DbConnection()
    {
        // Initialise database connection
        connection = new object(); // Represents actual DB connection
        Console.WriteLine("Database connection established.");
    }

    public static DbConnection GetInstance()
    {
        if (instance == null)
        {
            instance = new DbConnection();
        }
        return instance;
    }

    public object GetConnection()
    {

```

```
        return connection;
    }
}
```

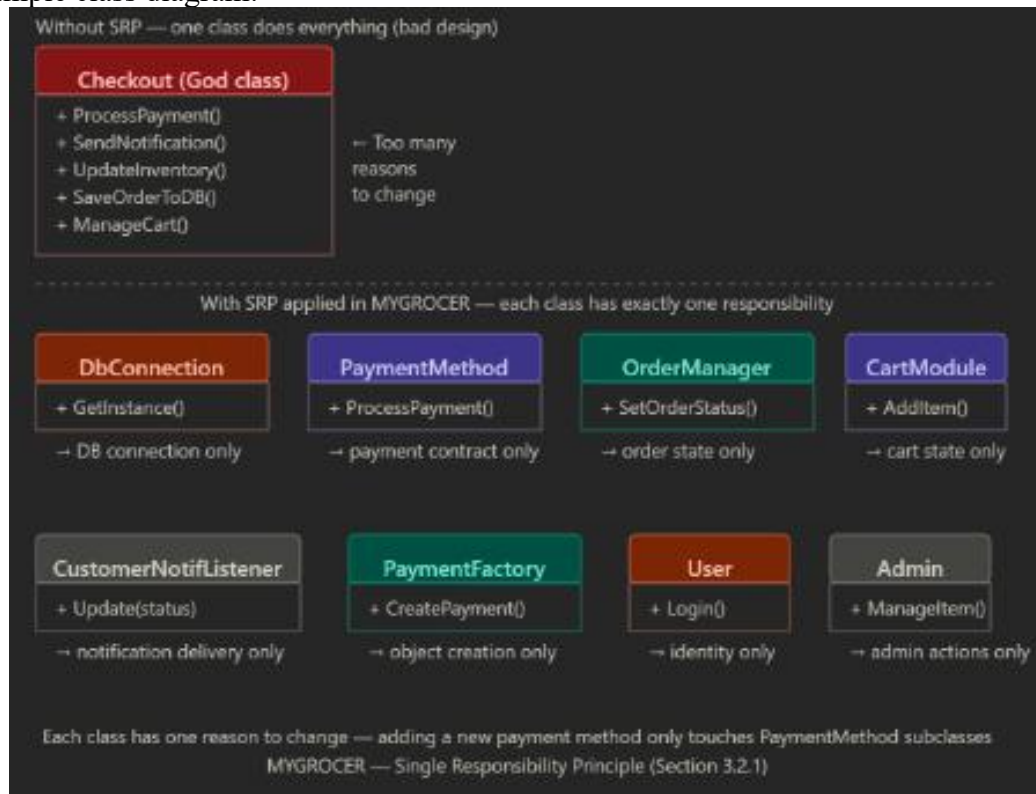
```
//SRP: OrderManager only manages order state and observer notification
class OrderManager
{
    private string orderStatus;

    public void SetOrderStatus(string status)
    {
        orderStatus = status;
        NotifyListeners();
    }

    public void NotifyListeners()
    {
        Console.WriteLine("Order status updated to: " + orderStatus);
    }
}
```

```
// SRP: CustomerNotifListener only handles customer notification delivery
class CustomerNotifListener
{
    public void Update(string status)
    {
        Console.WriteLine("Customer notified: Your order is now " + status);
    }
}
```

Sample class diagram:



Benefits:

- I. Each class is easier to understand, test, and maintain because it does only one thing.
- II. Reduces the risk of unintended side effects when modifying a class.
- III. Makes the codebase more modular and easier to extend.

3.2.2 Open / Closed Principle (OCP)

The Open/Closed Principle (OCP) basically means our code should be open for extension but closed for modification. In simpler terms, if we want to add new features, we should just write new code instead of messing with the existing code that already works perfectly fine.

How it is applied in MYGROCER: For MYGROCER, we applied this in the payment module. We have a base PaymentMethod class. If we decide to add a new payment type later, like Touch 'n Go eWallet, we don't have to touch the main Checkout code at all. We just create a new EWalletPayment class that inherits from the base class. This keeps the core system safe from new bugs.

Code Example 1: WITHOUT OCP (Violates the principle)

```
class Checkout
{
    // If we want to add E-Wallet or GrabPay later, we HAVE to modify this class
    // by adding more "else if" statements. This violates OCP.
    public void ProcessPayment(string paymentType)
    {
        if (paymentType == "FPX") {
            Console.WriteLine("Processing FPX...");
        }
        else if (paymentType == "Card") {
            Console.WriteLine("Processing Card...");
        }
        else if (paymentType == "COD") {
            Console.WriteLine("Processing COD...");
        }
    }
}
```

Code Example 2: WITH OCP (Follows the principle)

```
// --- [ CLOSED FOR MODIFICATION ] ---
// This core abstraction is locked in and doesn't need to change.
abstract class PaymentMethod
{
    public abstract void ProcessPayment();
}

// --- [ CLOSED FOR MODIFICATION ] ---
// The Checkout class is completely closed to changes. It will automatically
// work with ANY payment method we invent in the future.
class Checkout
{
```

```

    public void CompletePurchase(PaymentMethod method)
    {
        method.ProcessPayment();
    }
}

// Existing Features
class FPXPayment : PaymentMethod {
    public override void ProcessPayment() { Console.WriteLine("Processing FPX..."); }
}
class CardPayment : PaymentMethod {
    public override void ProcessPayment() { Console.WriteLine("Processing Card..."); }
}

// --- [ OPEN FOR EXTENSION ] ---
// We can extend the system's features by simply writing a NEW class.
// We didn't have to modify the Checkout class at all!
class EWalletPayment : PaymentMethod
{
    public override void ProcessPayment()
    {
        Console.WriteLine("Processing E-Wallet Payment...");
    }
}

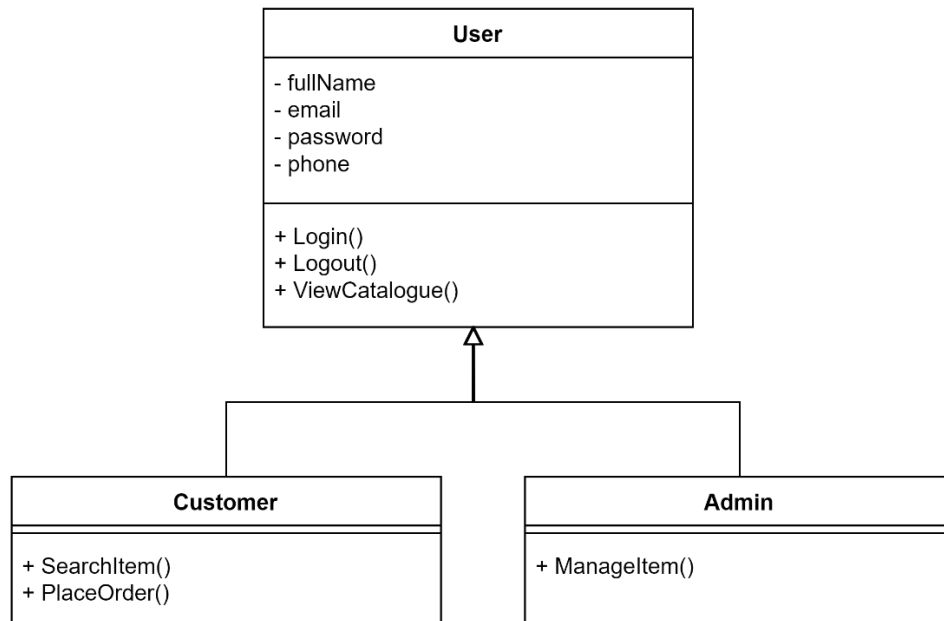
```

Benefits:

- It's much safer to add new features without breaking existing modules.
- We don't have to re-test the old code (like the checkout system) every time we add a new payment method.

3.2.3 Liskov Substitution Principle (LSP)

In MYGROCER, LSP is implemented for user management and authentication. Which consist of class User, Admin and Customer.



Both Customer and Admin objects can replace the User object without modifying the client code or affecting system functionality. This demonstrates the principle because all subclasses behave correctly when substituted for the parent class.

Benefits:

- Codes are loosely dependent to each other and encourages code reusability
- Reduce bugs by preventing unexpected behaviours that can occur from incompatible subclasses.
- Actions of the system is predictable because LRP guarantees consistent behaviours across different implementation.

Sample code WITHOUT LSP:

```
// Parent Class
class User
{
    public virtual void Login()
    {
        Console.WriteLine("User logged in.");
    }

    public virtual void ViewCatalogue()
    {
        Console.WriteLine("Viewing catalogue.");
    }
}

// Subclass
class Customer : User
{
    public override void Login()
    {
        Console.WriteLine("Customer logged in.");
    }
}

// BAD Subclass (Violates LSP)
class GuestUser : User
{
    public override void Login()
    {
        // Guest cannot login
        throw new Exception("Guest users cannot login.");
    }
}

// Client
class AuthenticationSystem
{
    public void AccessSystem(User user)
    {
        user.Login(); // expects every User can login
        user.ViewCatalogue();
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        AuthenticationSystem system = new AuthenticationSystem();

        User customer = new Customer();
        system.AccessSystem(customer);

        User guest = new GuestUser();
        system.AccessSystem(guest); // ERROR
    }
}
```

The system expects all user to be able to login. But GuessUser can't login and will throw an exception. Therefore, it cannot substitute and violate Liskov Substitution Principle.

Sample Code WITH LSP:

```
// Parent Class
class User
{
    public virtual void Login()
    {
        Console.WriteLine("User logged in.");
    }

    public virtual void Logout()
    {
        Console.WriteLine("User logged out.");
    }

    public virtual void ViewCatalogue()
    {
        Console.WriteLine("Viewing the catalogue.");
    }
}

// Subclass
class Customer : User
{
    public void SearchItem()
    {
        Console.WriteLine("Browsing grocery items.");
    }

    public void PlaceOrder()
    {
        Console.WriteLine("Order placed successfully.");
    }
}

// Subclass
class Admin : User
{
    public void ManageItem()
    {
        Console.WriteLine("Managing grocery items.");
    }

    public void ViewOrders()
    {
        Console.WriteLine("Viewing all orders.");
    }
}
```

```
// Client
class AuthenticationSystem
{
    public void AccessSystem(User user)
    {
        user.Login();
        user.ViewCatalogue();
    }
}

class Program
{
    static void Main(string[] args)
    {
        AuthenticationSystem system = new AuthenticationSystem();

        User customer = new Customer();
        User admin = new Admin();

        system.AccessSystem(customer);
        system.AccessSystem(admin);
    }
}
```

All subclasses(Admin and Customer) can login. Thus, all classes can substitute the parent class, User. This follows the LSP.

3.2.4 Interface Segregation Principle

The Interface Segregation Principle (ISP) is all about not forcing a class to implement methods it doesn't even need. Instead of having one massive interface that tries to do everything, it's better to break it down into smaller, specific ones.

How it is applied in MYGROCER:

If we didn't use ISP in MYGROCER, we might have a giant IUserActions interface. That would force the Customer class to have a ManageItem() method, even though customers can't manage inventory. Instead, we split them up. We made an IShoppingActions interface for the Customer and an IAdminActions interface for the Admin. This way, each role only gets the specific functions they actually use.

Sample Code (C#):

```
// Bad Design (Violates ISP): A fat interface
/*
interface IUserActions {
    void AddToCart();
    void Checkout();
    void ManageItem();
    void ViewAllOrders();
}
*/

// Good Design (Follows ISP): Segregated, role-specific interfaces
interface IShoppingActions
{
    void AddToCart();
    void Checkout();
}

interface IAdminActions
{
    void ManageItem();
    void ViewAllOrders();
}

// Customer only implements the interface relevant to shopping
class Customer : User, IShoppingActions
{
    public void AddToCart()
    {
        Console.WriteLine("Item added to the customer's cart.");
    }

    public void Checkout()
    {

```

```
        Console.WriteLine("Customer proceeding to checkout.");
    }
}

// Admin only implements the interface relevant to management
class Admin : User, IAdminActions
{
    public void ManageItem()
    {
        Console.WriteLine("Admin updating inventory stock.");
    }

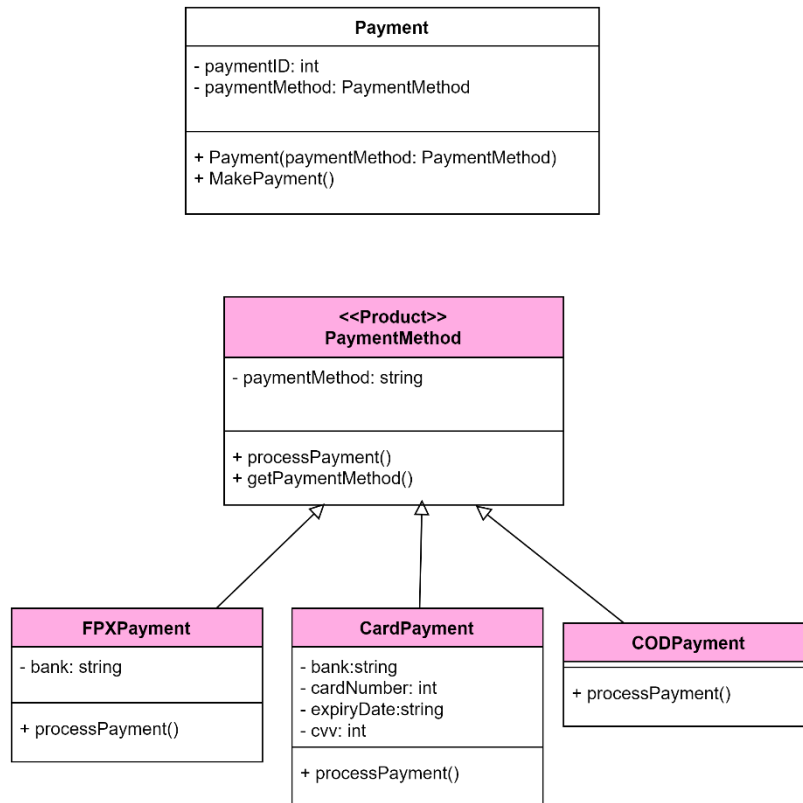
    public void ViewAllOrders()
    {
        Console.WriteLine("Admin viewing the system order list.");
    }
}
```

Benefits:

- Prevents messy, bloated classes that are hard to maintain.
- Makes the system more secure because a Customer object physically doesn't have access to admin-level methods.

3.2.5 Dependency Inversion Principle (DIP)

DIP is implemented during payment method selection in this system by making the Checkout class depends on the abstract PaymentMethod instead of the concrete payments (FPXPayment, CardPayment and CODPayment).



This design principle reduces tight coupling between the checkout system and payment gateways.

Benefits:

- Reduce coupling by depending on abstractions or interface this. This makes it easier to change or extend functionality.
- Can introduce new implementations of abstractions without changing the high-level classes, promoting code extensibility.

Sample Code WITHOUT DIP

In this code, the high-level Checkout class is dependent on the low-level classes like FPXPayment and CardPayment which are also concrete. This makes the system difficult to extend and maintain due to the tight coupling

```
// Low-level modules
class FPXPayment
{
    public void ProcessPayment()
    {
        Console.WriteLine("Processing FPX Payment...");
    }
}

class CardPayment
{
    public void ProcessPayment()
    {
        Console.WriteLine("Processing Card Payment...");
    }
}

// High-level module
class Payment
{
    public void MakePayment(string paymentType)
    {
        if(paymentType == "FPX")
        {
            FPXPayment payment = new FPXPayment();
            payment.ProcessPayment();
        }

        else if(paymentType == "CARD")
        {
            CardPayment payment = new CardPayment();
            payment.ProcessPayment();
        }
    }
}
```

```
// Client
class Program
{
    static void Main(string[] args)
    {
        Payment payment = new Payment();

        payment.MakePayment("FPX");
    }
}
```

Sample code WITH DIP

DIP improve maintainability and reduces coupling by making the Checkout class to depend on abstract class PaymentMethod.

```
// Client
class Program
{
    static void Main(string[] args)
    {
        Checkout checkout = new Checkout();

        checkout.MakePayment("FPX");
    }
}

// Abstraction
Abstract class PaymentMethod
{
    void ProcessPayment();
}

// Low-level modules
class FPXPayment : PaymentMethod
{
    public void ProcessPayment()
    {
        Console.WriteLine("Processing FPX Payment...");
    }
}
```

```

class CardPayment : PaymentMethod
{
    public void ProcessPayment()
    {
        Console.WriteLine("Processing Card Payment...");
    }
}

// High-level module
class Payment
{
    private PaymentMethod paymentMethod;

    public Payment (PaymentMethod paymentMethod)
    {
        this.paymentMethod = paymentMethod;
    }

    public void MakePayment()
    {
        paymentMethod.ProcessPayment();
    }
}

// Client
class Program
{
    static void Main(string[] args)
    {
        PaymentMethod paymentMethod = new FPXPayment();

        Payment payment = new Payment (paymentMethod);

        payment.MakePayment();
    }
}

```

4.0 Proposed Design Patterns

4.1 Singleton Pattern

i. Function or Software Component Affected

The Singleton Pattern is applied to the Database Connection layer of MYGROCER (DbConnection class). In a web-based e-commerce system like MYGROCER, multiple system modules including the AuthenticationModule, ProductModule, CartModule, OrderModule, and PaymentModule all require access to the database. Without a Singleton, each module could independently create its own database connection instance, leading to resource exhaustion, connection conflicts, and inconsistent data states.

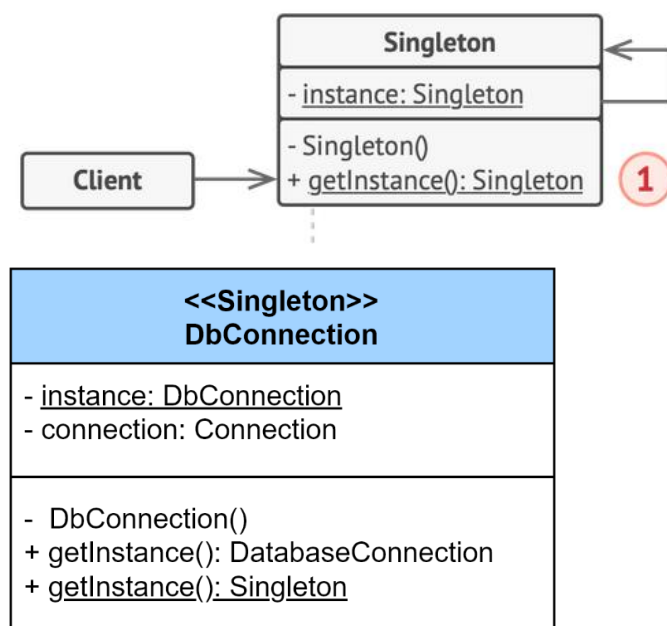
The DbConnection Singleton ensures that exactly one shared database connection instance is created and reused throughout the entire application lifecycle. Any module requesting a database connection receives the same instance via GetInstance().

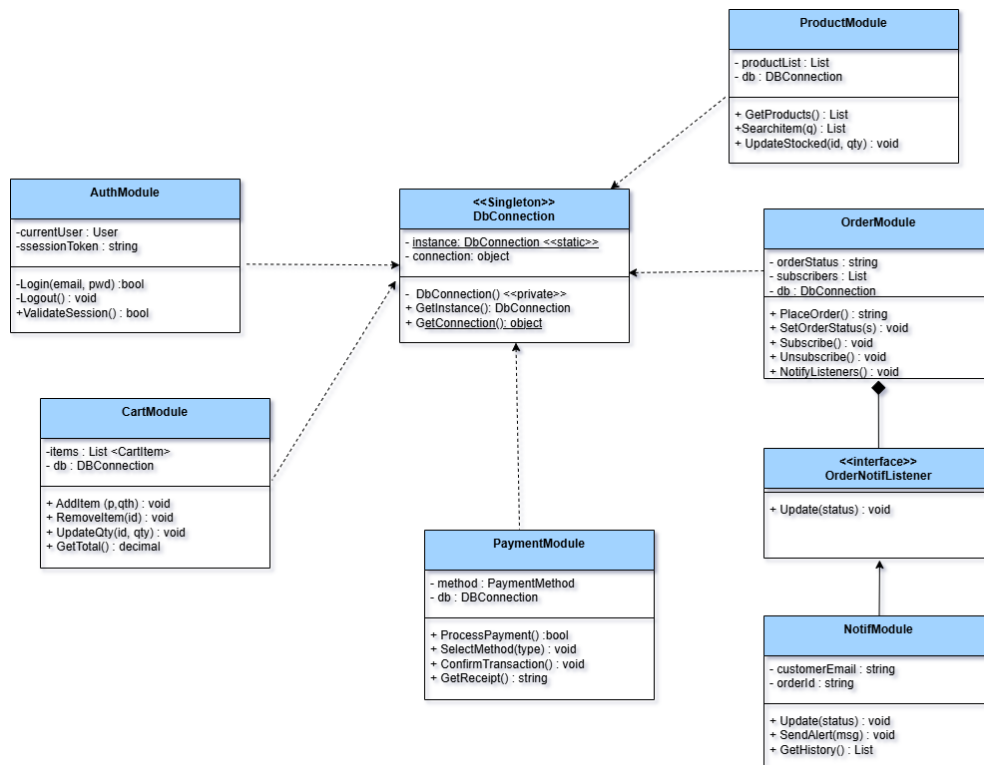
ii. Description of work flow or data flow

1. A system module (e.g., ProductModule) needs to query the database.
2. It calls DbConnection.GetInstance().
3. If no instance exists yet, the DbConnection constructor runs once, establishing the physical database connection and storing the instance in the private static instance variable.
4. If an instance already exists, GetInstance() returns the existing instance immediately no new connection is created.
5. The module calls GetConnection() on the returned instance to retrieve the connection object and execute its database operations.
6. All subsequent modules that call GetInstance() receive the same connection instance.

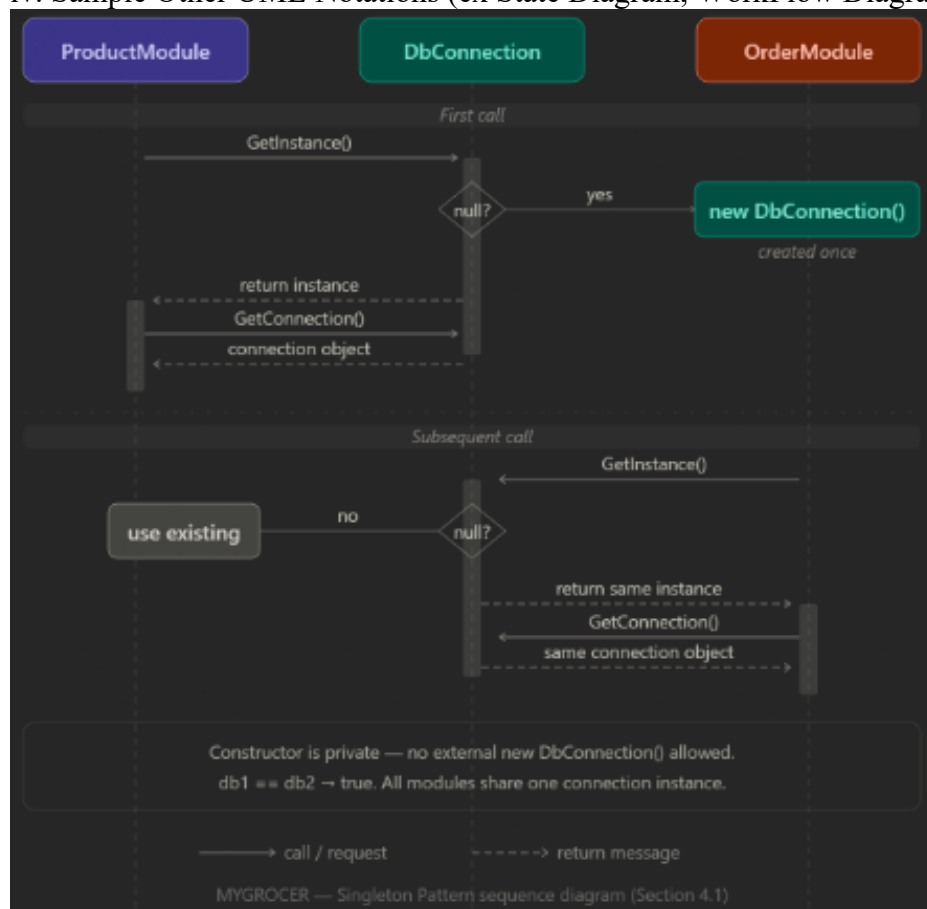
Key rule: The constructor is private, so no external code can call new DbConnection() directly. The only entry point is GetInstance().

iii. Sample Class Diagram





iv. Sample Other UML Notations (ex State Diagram, WorkFlow Diagram)



v. Sample Code (C#) Before Using Singleton Pattern:

```
// BEFORE Singleton: Every module creates its own database connection
// Problem: 5 modules = 5 separate connections open simultaneously.
// Risk of connection conflicts, resource exhaustion, and inconsistent data states.

class ProductModule
{
    public void LoadProducts()
    {
        // Creates a brand new connection — unaware others already exist
        object connection = new object();
        Console.WriteLine("ProductModule: New DB connection opened.");
        Console.WriteLine("ProductModule: Loading products...");
        // Connection is NOT shared — it is a separate instance
    }
}

class OrderModule
{
    public void SaveOrder()
    {
        // Opens yet another separate connection
        object connection = new object();
        Console.WriteLine("OrderModule: New DB connection opened.");
        Console.WriteLine("OrderModule: Saving order...");
    }
}

class CartModule
{
    public void SaveCart()
    {
        object connection = new object();
        Console.WriteLine("CartModule: New DB connection opened.");
        Console.WriteLine("CartModule: Saving cart data...");
    }
}

class Program
{
    static void Main(string[] args)
    {
        ProductModule products = new ProductModule();
        OrderModule orders = new OrderModule();
        CartModule cart = new CartModule();

        products.LoadProducts(); // Opens connection #1
        orders.SaveOrder();      // Opens connection #2
        cart.SaveCart();         // Opens connection #3
    }
}
```

```

// 3 separate connections now open simultaneously.
// Under high traffic (100 users), this creates 300+ open connections.
// No guarantee they read/write consistent data.
}
}

```

Expected Output:

```

ProductModule: New DB connection opened.
ProductModule: Loading products...
OrderModule: New DB connection opened.
OrderModule: Saving order...
CartModule: New DB connection opened.
CartModule: Saving cart data...

```

Problem: Three different connection objects exist at the same time. Compare this to the Singleton output where `Same instance?` `True` confirms only one connection ever exists.

vi. Sample Potential Code (C#) After Using Singleton Pattern:

```

using System;

// Singleton: DbConnection
// Ensures only one database connection instance exists across the entire system
class DbConnection
{
    // Step 1: Private static variable holds the single instance
    private static DbConnection instance = null;

    // Step 2: Represents the actual database connection object
    private object connection;

    // Step 3: Private constructor - prevents external instantiation with 'new'
    private DbConnection()
    {
        // Simulates establishing a database connection
        connection = new object();
        Console.WriteLine("Database connection established (single instance created).");
    }

    // Step 4: Public static method - the ONLY way to access the instance
    public static DbConnection GetInstance()
    {
        if (instance == null)
        {
            instance = new DbConnection(); // Created only once
        }
        return instance;
    }
}

```

```

// Step 5: Returns the connection object for use by modules
public object GetConnection()
{
    return connection;
}
}

// Client: ProductModule uses DbConnection
class ProductModule
{
    public void LoadProducts()
    {
        DbConnection db = DbConnection.GetInstance();
        object conn = db.GetConnection();
        Console.WriteLine("ProductModule: Loading products using shared DB
connection.");
        // Execute: SELECT * FROM Products
    }
}

// Client: OrderModule also uses the same DbConnection
class OrderModule
{
    public void SaveOrder()
    {
        DbConnection db = DbConnection.GetInstance(); // Returns the SAME
instance
        object conn = db.GetConnection();
        Console.WriteLine("OrderModule: Saving order using shared DB
connection.");
        // Execute: INSERT INTO Orders ...
    }
}

// Program: Entry point demonstrating single instance behaviour
class Program
{
    static void Main(string[] args)
    {
        ProductModule products = new ProductModule();
        OrderModule orders = new OrderModule();

        products.LoadProducts();
        orders.SaveOrder();

        // Prove it is the same instance
        DbConnection db1 = DbConnection.GetInstance();
        DbConnection db2 = DbConnection.GetInstance();
        Console.WriteLine("Same instance? " + (db1 == db2)); // Output: True
    }
}

```

```
}  
}
```

Expected Output:

```
Database connection established (single instance created).  
ProductModule: Loading products using shared DB connection.  
OrderModule: Saving order using shared DB connection.  
Same instance? True
```

vii. Benefit

Resource Efficiency: Only one database connection is maintained throughout the application, preventing resource exhaustion from multiple simultaneous connections.

- a. Consistency: All system modules operate on the same connection state, eliminating synchronisation issues or conflicting transactions.
- b. Global Access Point: Any module in the system can access the database connection without needing it passed as a parameter, reducing dependency injection complexity.
- c. Controlled Instantiation: The private constructor completely prevents accidental creation of duplicate instances, enforcing the single-instance rule at compile time.
- d. Lazy Initialisation: The connection is only created when first requested, not at application startup, saving resources if certain modules are unused.

ix. Limitations

- Single Point of Failure: If the Singleton DbConnection instance encounters an error or becomes corrupted, all modules that depend on it are affected simultaneously.
- Difficult to Unit Test: Because the Singleton manages global state, unit testing individual modules in isolation becomes harder tests can interfere with each other through the shared instance.
- Not Thread-Safe by Default: In a concurrent environment (multiple simultaneous user requests), two threads could both pass the `if (instance == null)` check simultaneously and create two instances. A thread-safe lock mechanism must be added:

```
// Thread-safe Singleton (for concurrent environments)
private static readonly object lockObj = new object();

public static DbConnection GetInstance()
{
    if (instance == null)
    {
        lock (lockObj)
        {
            if (instance == null)
            {
                instance = new DbConnection();
            }
        }
    }
    return instance;
}
```

- Tight Global Coupling: Since the Singleton is accessed globally, many modules become implicitly dependent on it, making the system harder to refactor if the database layer needs to change.
- Violates SRP if Extended: If additional responsibilities (e.g., connection pooling, query logging) are added to DbConnection, it risks violating the Single Responsibility Principle.

4.2 Factory Pattern

i. Function or Software Component Affected

Factory Method is applied for product management in the system. This design pattern is used to create different payment method objects without exposing the object creation logic. This pattern is affective during checkout, payment processing and order management.

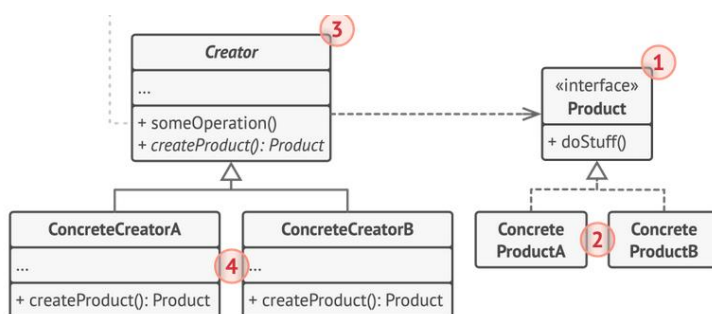
It is implemented by focusing on the PaymentMethod class and its subclasses, FPXPayment, CardPayment, CODPayment and EWalletPayment. Moreover, this design also makes it easier to add future payment method without modifying the existing code.

ii. Description of workflow or data flow

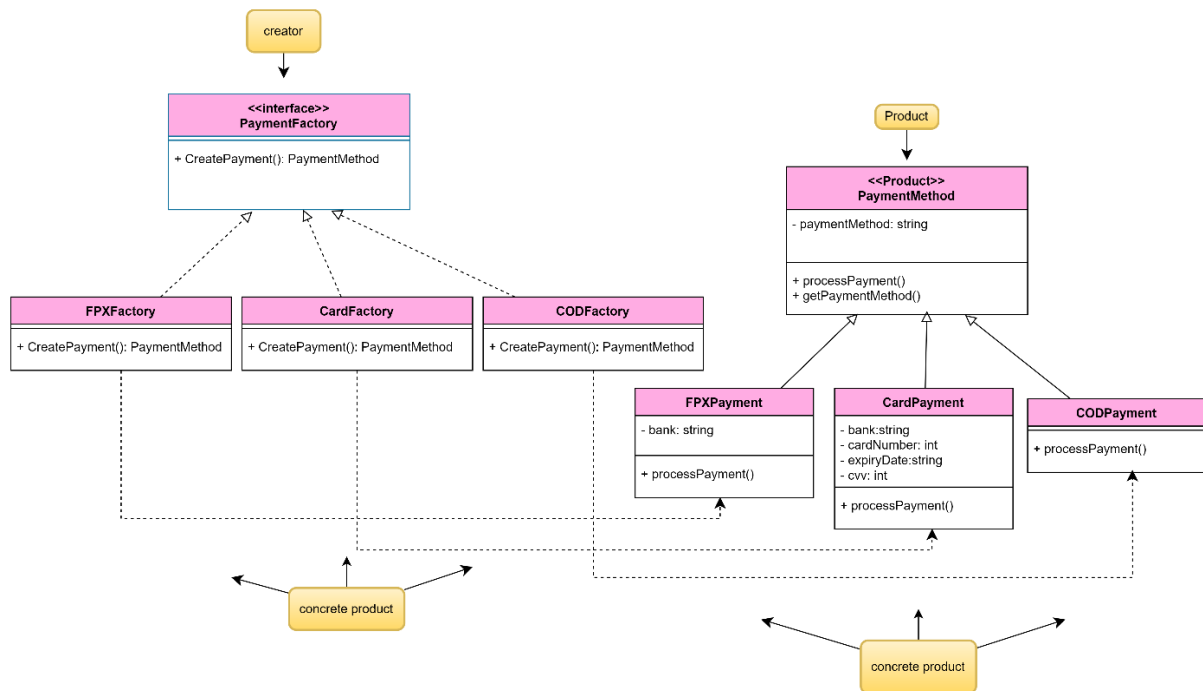
1. Customer proceeds to checkout.
2. Customer choose a payment method.
3. Selected payment type is sent to the PaymentFactory class.
4. The PaymentFactory determines which payment object should be created.
5. The corresponding payment object is instantiated:
 - FPXPayment
 - CardPayment
 - CODPayment
6. The payment object processes the payment transaction.
7. The system returns the payment result and updates the order status.

iii. Sample Class Diagram

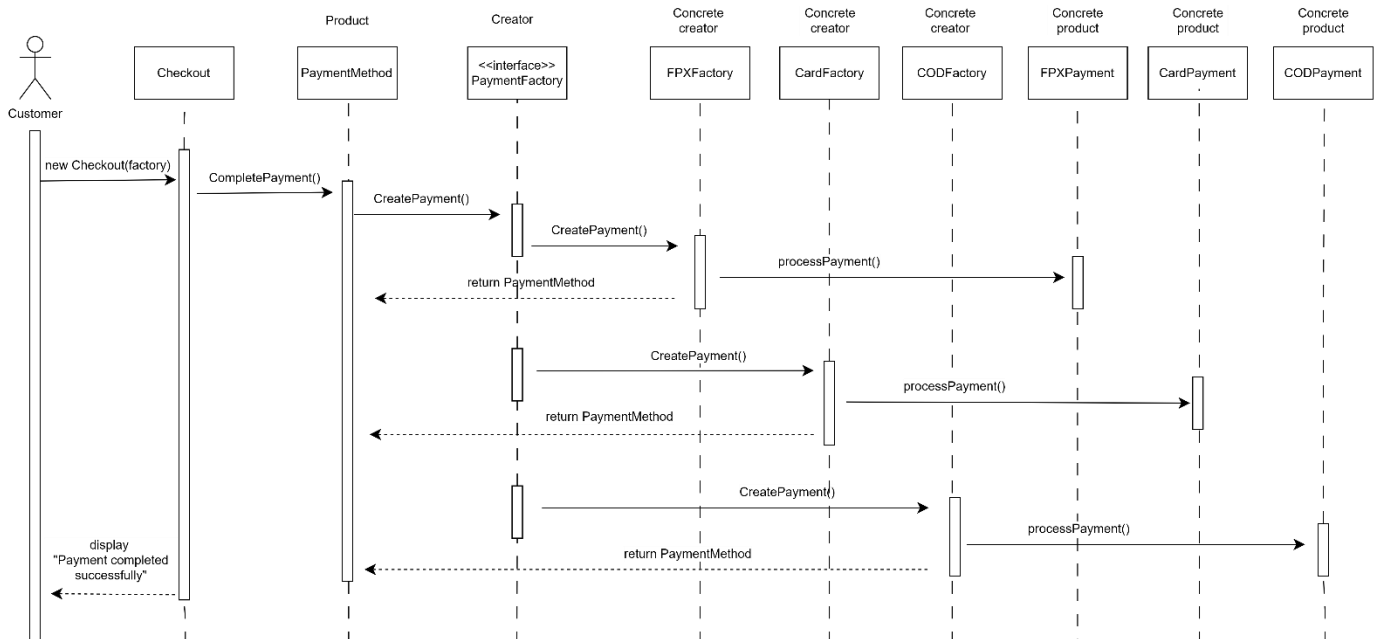
Original



System



iv. Sample Other UML Notations (ex State Diagram, WorkFlow Diagram)



v. Sample Potential Code

```
// Product
abstract class PaymentMethod
{
    public abstract void ProcessPayment();
}

// Concrete Products
class FPXPayment : PaymentMethod
{
    public override void ProcessPayment()
    {
        Console.WriteLine("Processing FPX Payment...");
    }
}
class CardPayment : PaymentMethod
{
    public override void ProcessPayment()
    {
        Console.WriteLine("Processing Card Payment...");
    }
}
class CODPayment : PaymentMethod
{
    public override void ProcessPayment()
    {
        Console.WriteLine("Processing Cash On Delivery...");
    }
}

// Creator
abstract class PaymentFactory
{
    public abstract PaymentMethod CreatePayment();
}

// Concrete Creators
class FPXFactory : PaymentFactory
{
    public override PaymentMethod CreatePayment()
    {
        return new FPXPayment();
    }
}
```

```

class CardFactory : PaymentFactory
{
    public override PaymentMethod CreatePayment()
    {
        return new CardPayment();
    }
}

class CODFactory : PaymentFactory
{
    public override PaymentMethod CreatePayment()
    {
        return new CODPayment();
    }
}

class Checkout
{
    private PaymentFactory factory;

    public Checkout(PaymentFactory factory)
    {
        this.factory = factory;
    }

    public void CompletePayment()
    {
        PaymentMethod payment = factory.CreatePayment();
        payment.ProcessPayment();

        Console.WriteLine("Payment completed successfully.");
    }
}

class Program
{
    static void Main(string[] args)
    {
        // Example: Customer selects FPX payment
        PaymentFactory factory = new FPXFactory();

        Checkout checkout = new Checkout(factory);

        checkout.CompletePayment();
    }
}

```

vi. Benefit

- Loose Coupling: Reduces tight coupling between checkout logic and payment implementations.
- Flexibility and Scalability: New payment methods can be added without modifying existing code. System is more flexible for maintenance and future integrations such as Touch 'n Go eWallet or GrabPay.
- Encapsulation: Each payment method is encapsulated in its class, making the system easier to manage and maintain.

vii. Limitations

- Increases the number of classes in the system.
- Factory logic may become more complex as more payment methods are added.
- Additional abstraction may slightly increase development complexity.
- Debugging may be harder due to indirect object creation; the flow of object creation is hard to trace

4.3 Observer Pattern

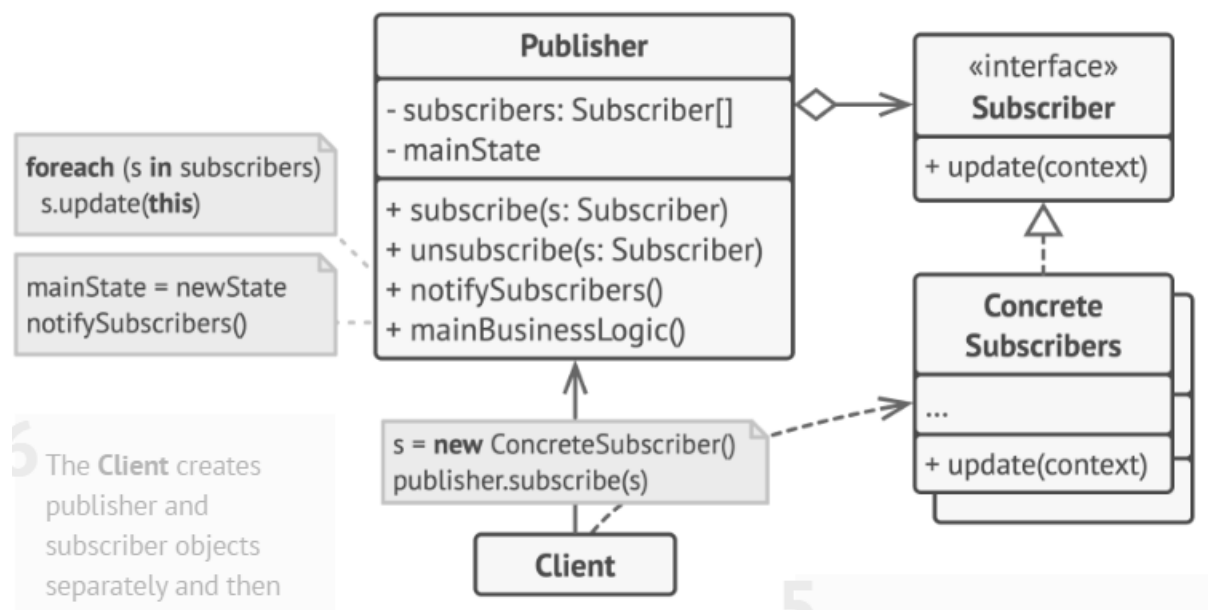
i. Function or Software Component Affected

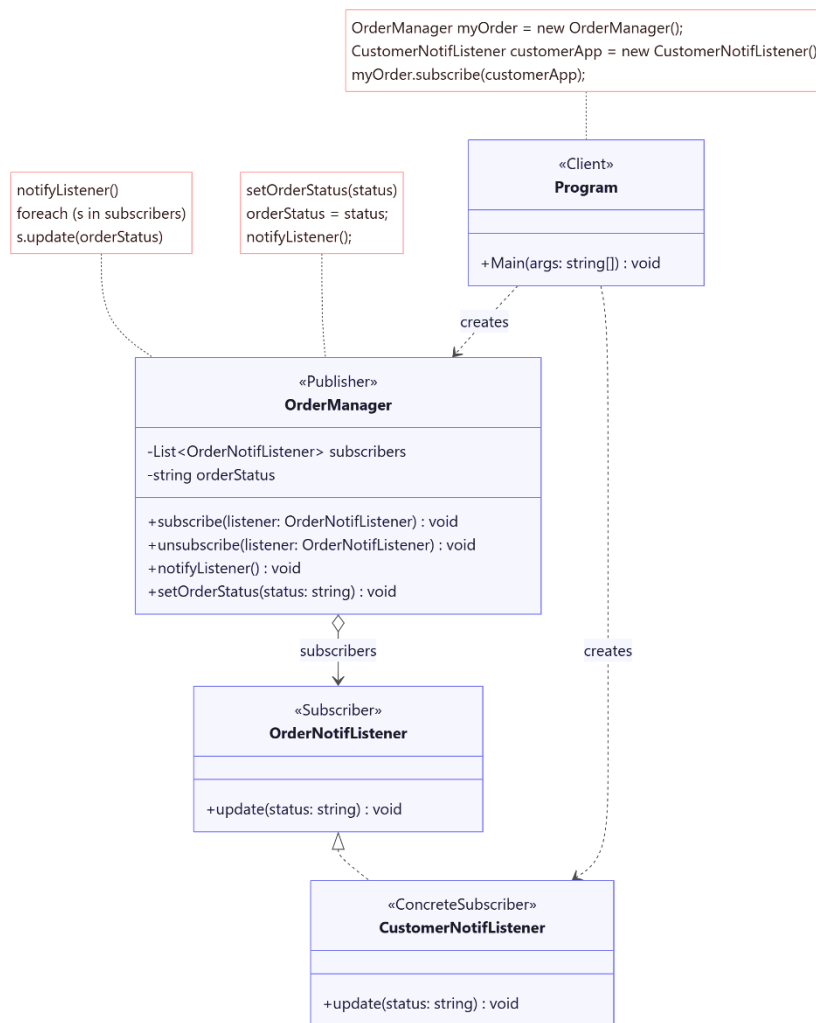
We applied the Observer Pattern to the Order Management and Notification module in MYGROCER. Basically, instead of the customer having to constantly refresh the page to see if their groceries are on the way, the system automatically pushes an update to them the second the status changes. It primarily affects the OrderManager class (which acts as the Publisher) and listener classes like CustomerNotifListener (which act as the Subscribers).

ii. Description of work flow or data flow

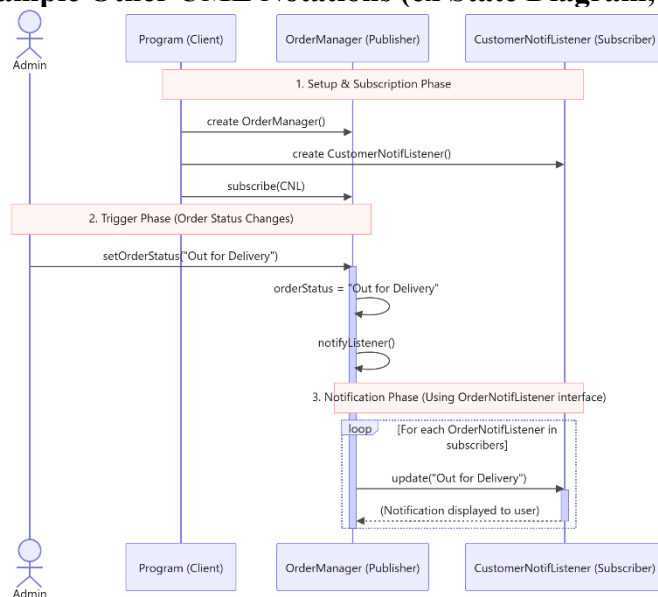
This sequence diagram illustrates the Observer Pattern for **Order Notifications**. It involves exactly the classes from our class diagram: The Program (Client) acts as the setup, the OrderManager acts as the Publisher, and the CustomerNotifListener acts as the Concrete Subscriber (which implements the OrderNotifListener interface).

iii. Sample Class Diagram





iv. Sample Other UML Notations (ex State Diagram, WorkFlow Diagram)



v. Sample Potential Code

```
using System;
using System.Collections.Generic;

// Subscriber Interface
interface OrderNotifListener
{
    void update(string status);
}

// Concrete Subscriber (Handles notifications for the Customer)
class CustomerNotifListener : OrderNotifListener
{
    public void update(string status)
    {
        // In the actual app, this pushes an alert to the frontend UI
        Console.WriteLine("Notification sent to Customer: Your order is now " + status + ".");
    }
}

// Publisher
class OrderManager
{
    // List to keep track of all subscribers
    private List<OrderNotifListener> subscribers = new List<OrderNotifListener>();
    private string orderStatus;

    public void subscribe(OrderNotifListener listener)
    {
        subscribers.Add(listener);
    }

    public void unsubscribe(OrderNotifListener listener)
    {
        subscribers.Remove(listener);
    }

    public void setOrderStatus(string status)
    {
        orderStatus = status;
        // The magic happens here: automatically notify everyone when the status changes
        notifyListener();
    }

    public void notifyListener()
    {
        foreach (var s in subscribers)
```

```

    {
        s.update(orderStatus); // Passes the new context to the subscriber
    }
}

// Client / Simulation
class Program
{
    static void Main(string[] args)
    {
        OrderManager myOrder = new OrderManager();
        CustomerNotifListener customerApp = new CustomerNotifListener();

        // Customer's app subscribes to the order
        myOrder.subscribe(customerApp);

        // Admin updates the status during fulfilment
        myOrder.setOrderStatus("Processing");
        myOrder.setOrderStatus("Out for Delivery");
        myOrder.setOrderStatus("Delivered");

        // Unsubscribe once delivered so we don't keep them in memory
        myOrder.unsubscribe(customerApp);
    }
}

```

vi. Benefit

1. Loose Coupling: The OrderManager doesn't need to know *how* the customer gets notified (whether it's an email, SMS, or web popup). It just yells "hey, the status changed!" and the listener handles the rest.
2. Real-Time Updates: It ensures customers are always looking at the most up-to-date order status without the system needing to run heavy, constant database checks.

vii. Limitations

1. Memory Leaks (Lapsed Listener Problem): If we forget to write code to unsubscribe() a customer after their grocery order is completed, the system keeps them in memory forever, which could eventually slow down the server.
2. Notification Spam: If an admin accidentally clicks the wrong status button a few times in a row, the observer pattern will blindly fire off a notification for every single click, which could annoy the customer.

5. Conclusion and suggestion

5.1 Conclusion

Building the MYGROCER e-commerce system has been a successful step toward making grocery shopping in Malaysia more digital and accessible. We managed to take the affordable hypermarket experience and turn it into a functional web platform, bridging the gap between physical stores and online convenience for urban consumers.

On the technical side, we built the system's architecture using solid Software Engineering concepts and SOLID principles. By sticking to rules like the Single Responsibility Principle (SRP) and Dependency Inversion Principle (DIP), we kept the code modular and easy to manage. We also applied key Design Patterns exactly where they made the most sense: the Singleton pattern keeps our database connections safe, the Factory pattern makes adding new payment methods easy, and the Observer pattern handles our real-time order tracking. Overall, this project shows how theoretical software design concepts can actually be used to build a practical, working system.

5.2 Suggestions for Future Enhancement

While MYGROCER currently does what it was built to do, there is always room for improvement. If we were to continue developing the system, here are a few upgrades we would suggest:

- **Developing a Mobile App:** Moving from a web-based system to a dedicated mobile app (for Android and iOS) would make it much easier for customers to shop on the go. It would also let us use native phone features like push notifications for order updates.
- **Integrating Third-Party Delivery APIs:** Right now, admins update the delivery status manually. Integrating with delivery services like Lalamove or GrabExpress would give customers real-time GPS tracking and save the admin team a lot of manual work.
- **Adding Smart Product Recommendations:** We could implement a basic AI feature that tracks what users usually buy. It could suggest

"Frequently Bought Together" items or cheaper alternatives, which would make shopping faster and help increase sales.

- **Expanding E-Wallet Options:** Since we already set up the Factory pattern for payments, it would be very easy to add direct API connections for popular local e-wallets like Touch 'n Go, Boost, or Apple Pay to give users more checkout choices.
- **Creating a Loyalty Points System:** Just like real hypermarkets, we could add a "MYGROCER Points" system. Customers earning points for every purchase would encourage them to keep coming back and buying their groceries in bulk.

6. Bibliography/Reference

H. Prabowo, E. N. Hindarwati and Yuniarty, "Online Grocery Shopping Adoption: A Systematic Literature Review," 2020 International Conference on Information Management and Technology (ICIMTech), Bandung, Indonesia, 2020, pp. 40-45, <https://doi.org/10.1109/ICIMTech50083.2020.9211241> .

Moorthy, K., Lim, S. X., Normala S.Govindarajo, & Loh, C. T. (2025). A study of online grocery shopping behaviour in Malaysia. *Issues and Perspectives in Business and Social Sciences*, 5(1), 1–14. <https://doi.org/10.33093/ipbss.2025.5.1.1>

Eriksson, N., & Stenius, M. (2023). What do regular online grocery shoppers want from online grocers going forward? Suggestions for service quality improvements. *Procedia Computer Science*, 219, 201–210. <https://doi.org/10.1016/j.procs.2023.01.282>

Moorthy, M. K., Lim, S. X., Govindarajo, N. S., & Loh, C. T. (2025). *A study of online grocery shopping behaviour in Malaysia*. ResearchGate. https://www.researchgate.net/publication/388102546_A_study_of_online_grocery_shopping_behaviour_in_Malaysia

ScienceDirect Article [PII: S1877050923002909]. (n.d.). Elsevier. <https://www.sciencedirect.com/science/article/pii/S1877050923002909>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.

Martin, R. C. (2002). *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall.